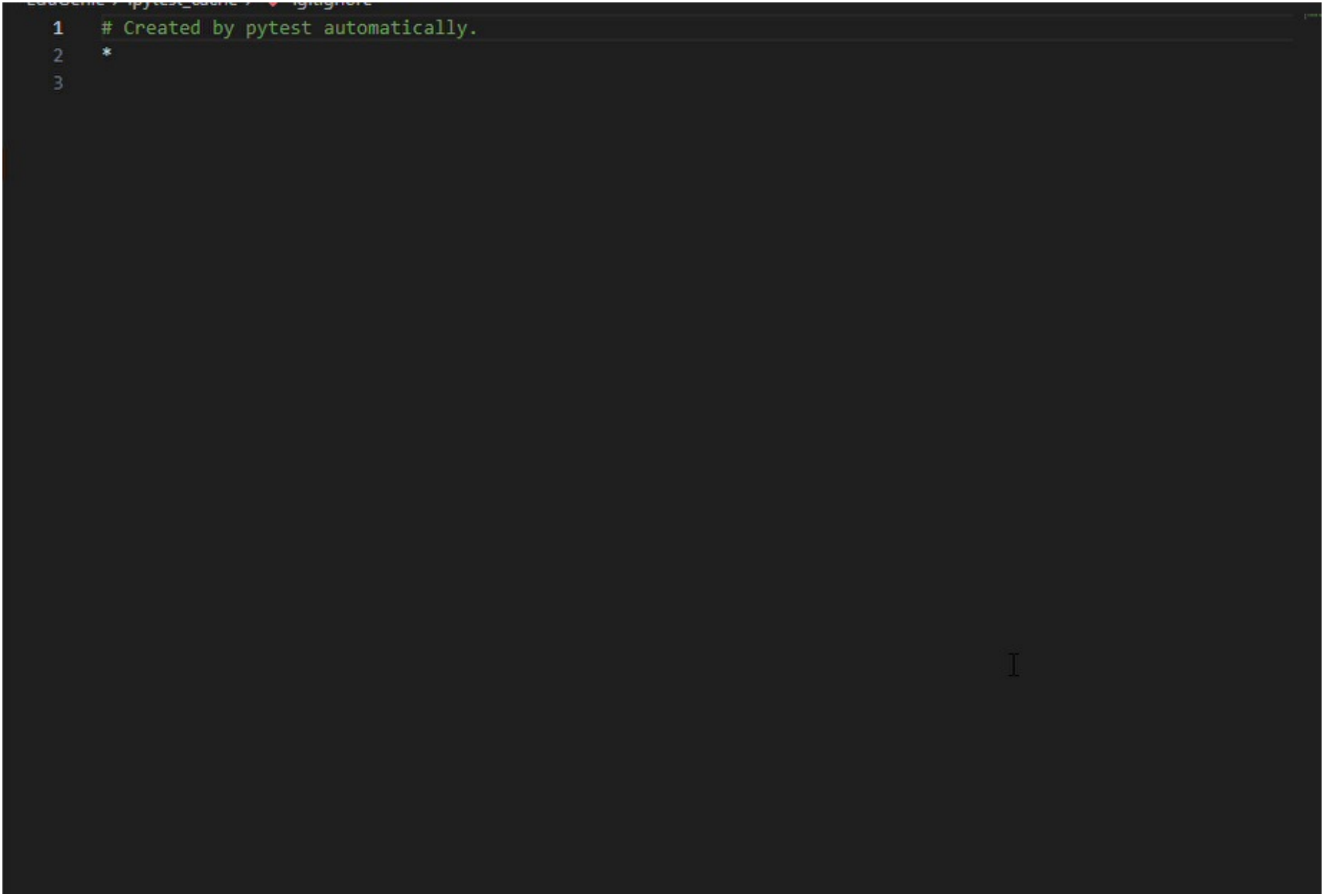
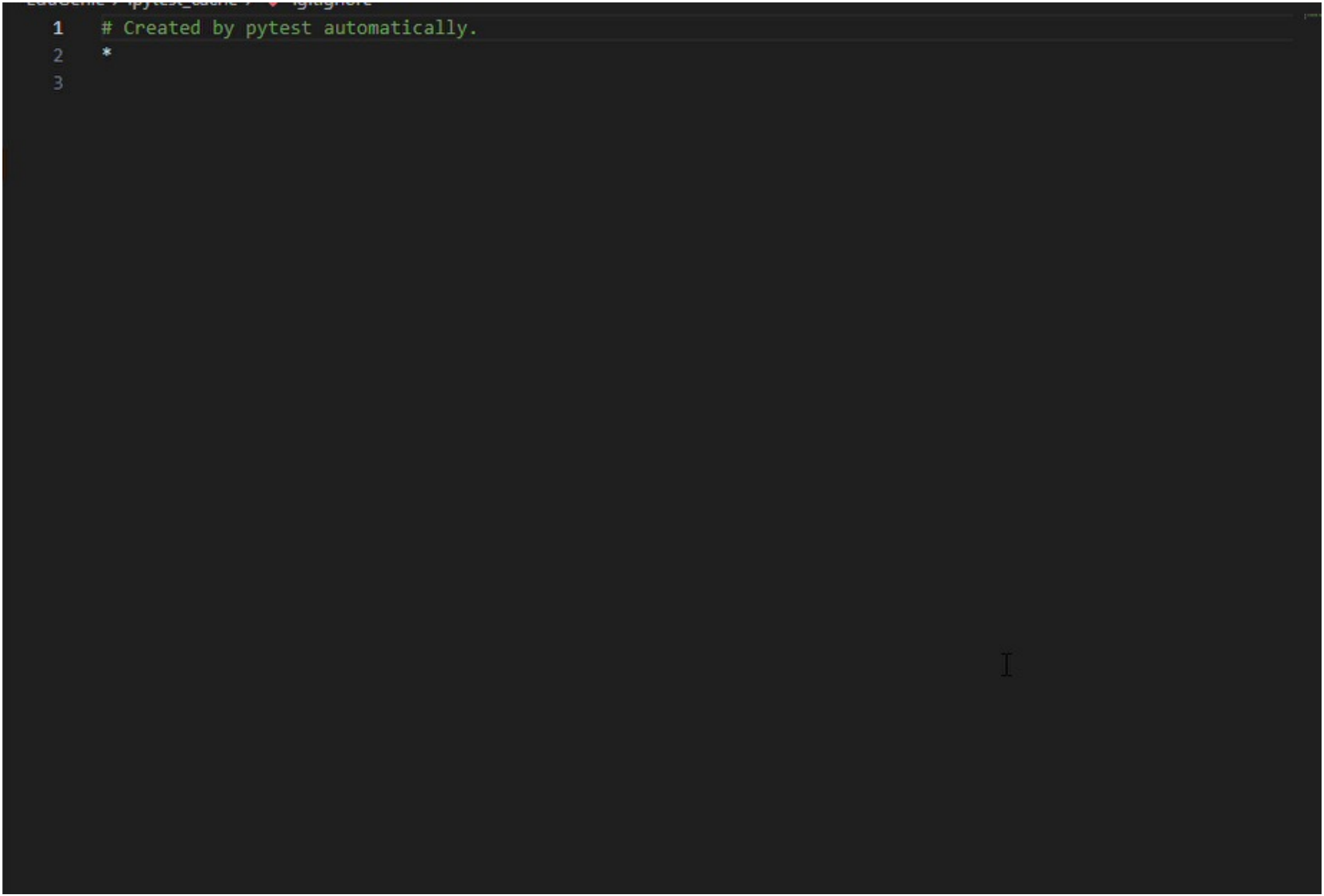
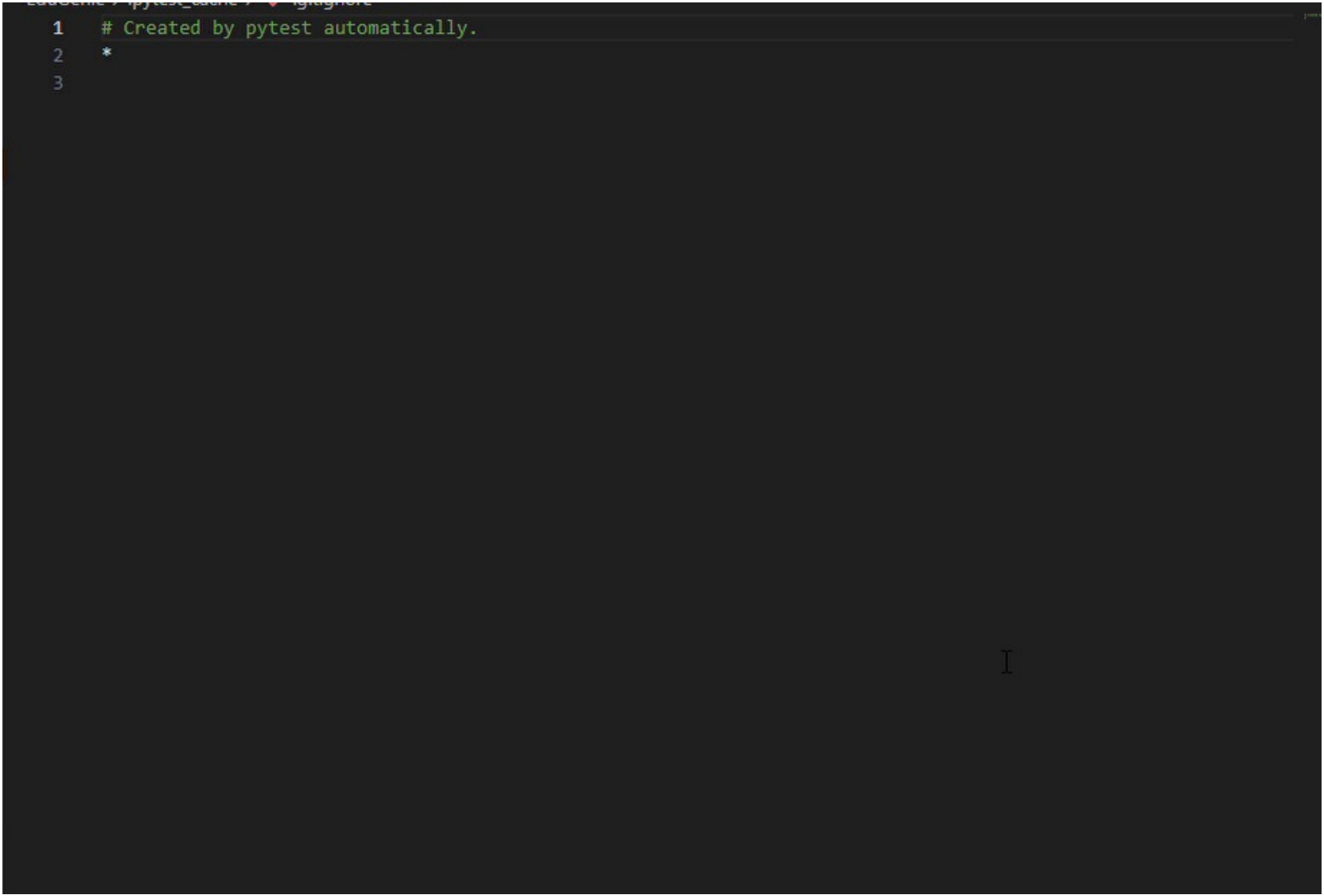
Code shown in video — timestamp 00:00



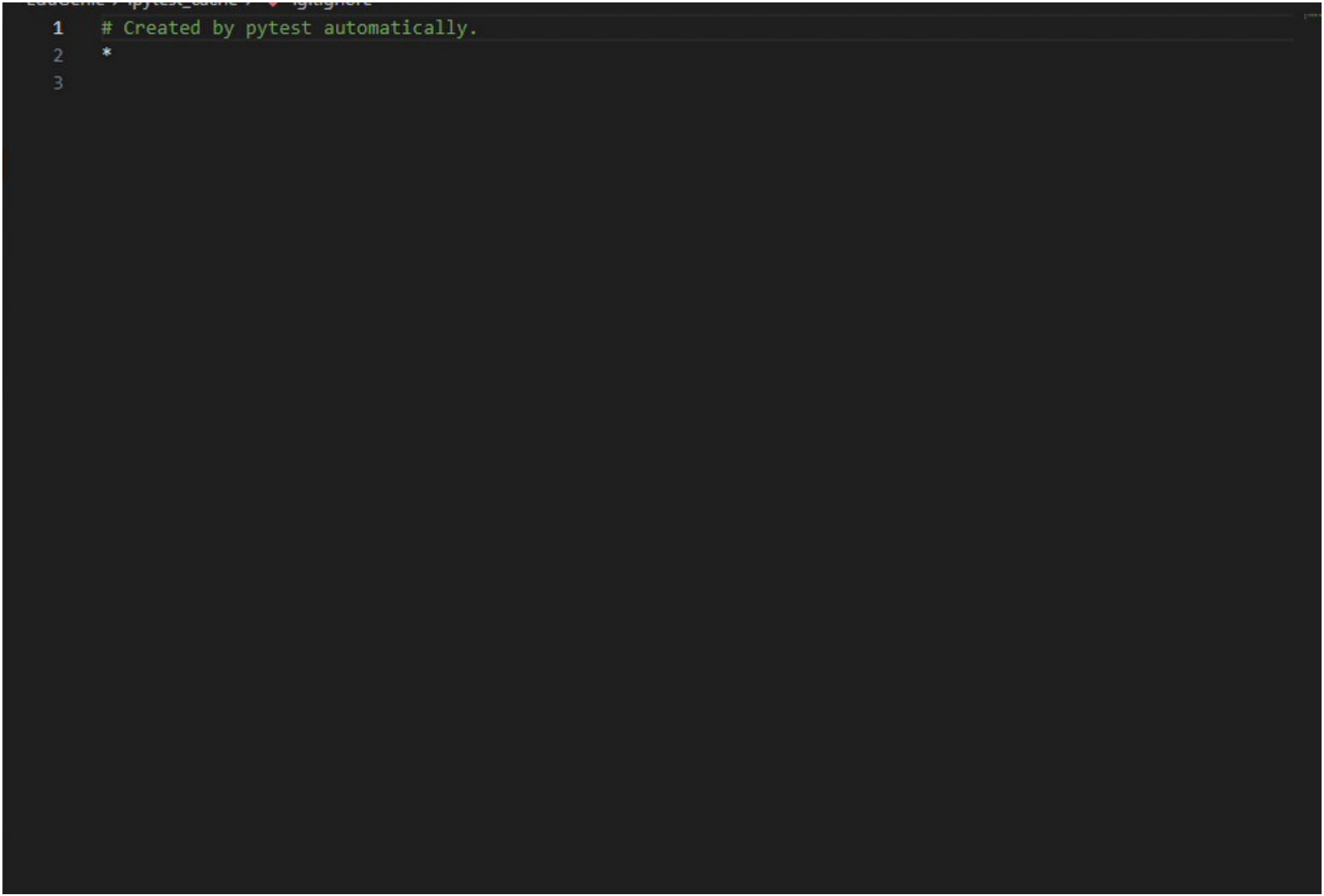
Code shown in video — timestamp 00:01



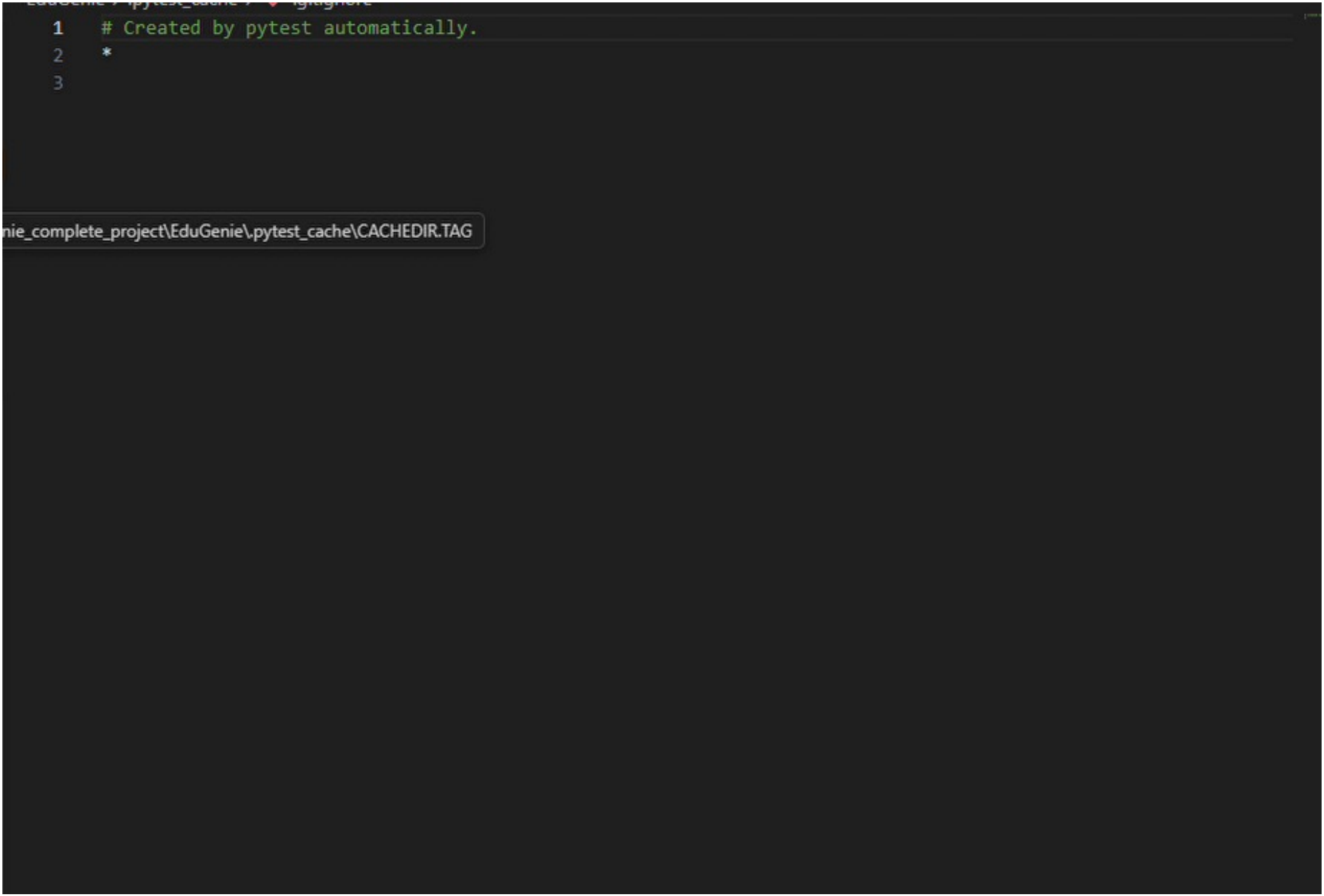
Code shown in video — timestamp 00:03



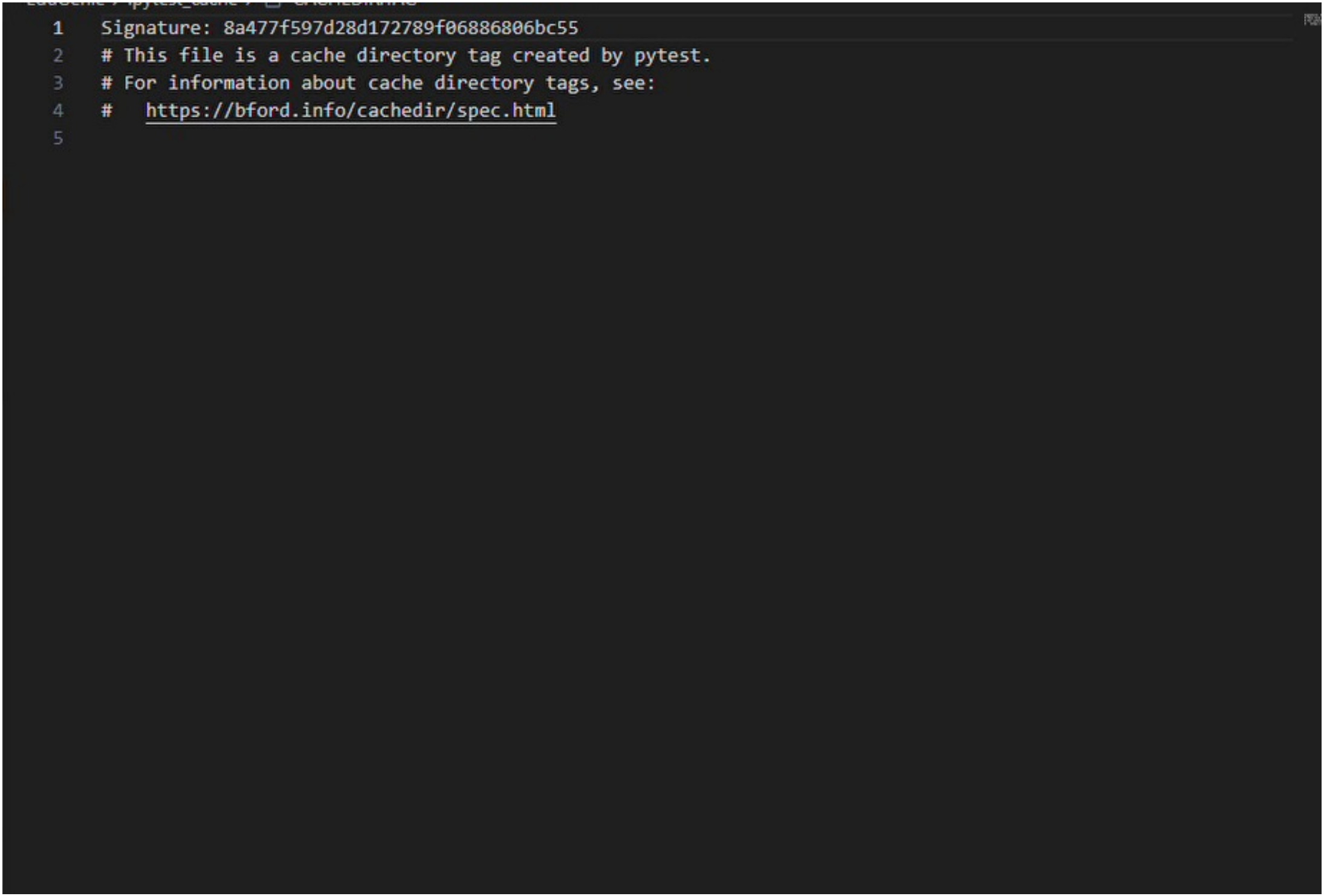
Code shown in video — timestamp 00:04



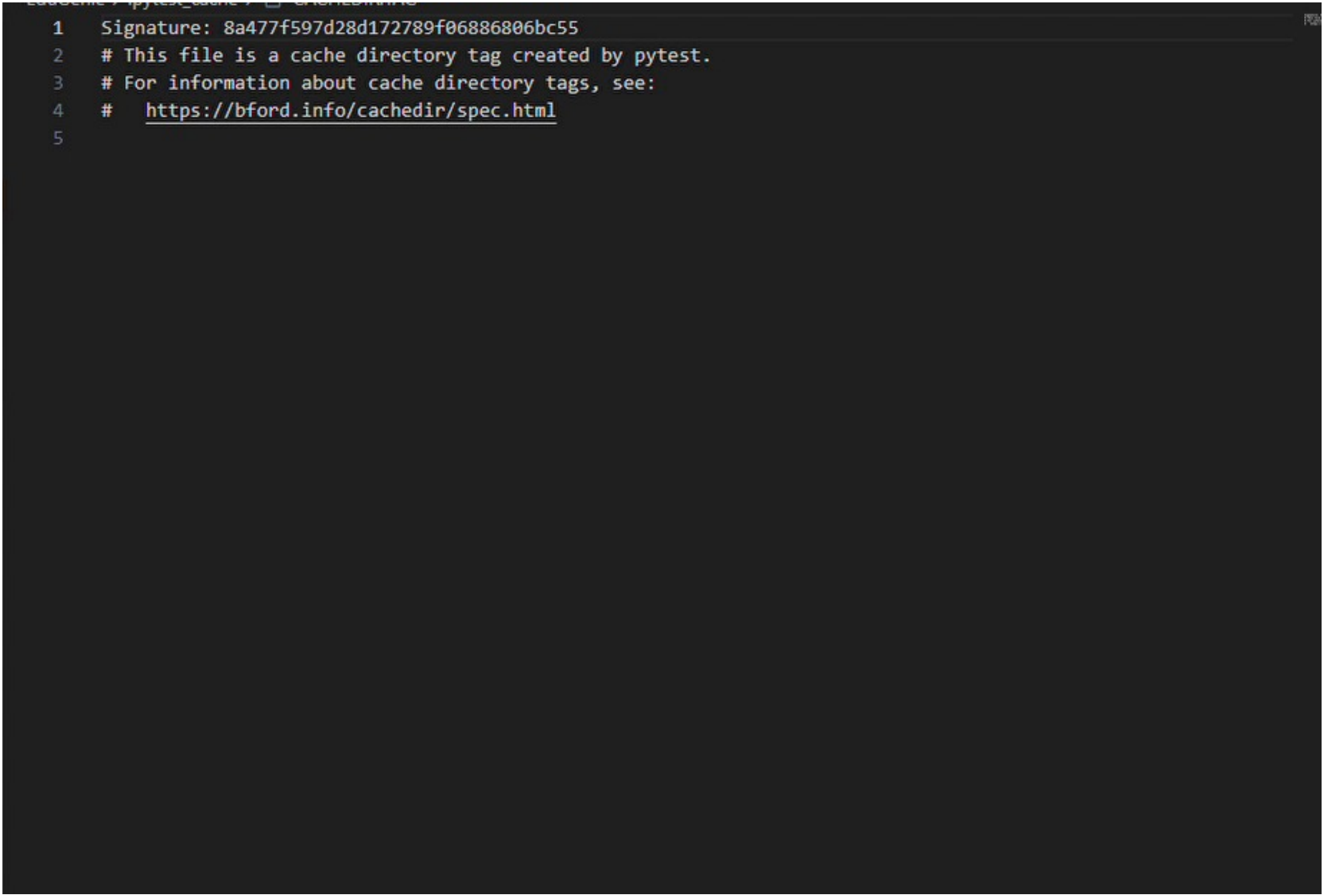
Code shown in video — timestamp 00:06



Code shown in video — timestamp 00:07



Code shown in video — timestamp 00:09



Code shown in video — timestamp 00:10

```
# pytest cache directory #

This directory contains data from the pytest's cache plugin,
which provides the `--lf` and `--ff` options, as well as the `cache` fixture.

**Do not** commit this to version control.

See [the docs](https://docs.pytest.org/en/stable/how-to/cache.html) for more information.

```

Code shown in video — timestamp 00:12

```
    background: rgba(255,255,255,.018);
  }
  .mini-grid span { color: #53677f; font-size: 10px; }
  .mini-grid strong { display: block; margin-top: 18px; font: 600 14px "Space Grotesk"; }
  .mini-grid p { color: #718399; font-size: 12px; line-height: 1.5; margin-bottom: 0; }

  footer { text-align: center; color: #4f637b; font-size: 11px; padding: 10px 20px 35px; }

  @media (max-width: 850px) {
    .workspace { grid-template-columns: 1fr; }
    .panel { min-height: auto; }
    .mini-grid { grid-template-columns: 1fr 1fr; }
  }
  @media (max-width: 560px) {
    .topbar { padding: 18px; }
    .status { display: none; }
    .shell { padding: 10px 15px 35px; }
    .hero { padding: 40px 0 28px; }
    .hero h1 { letter-spacing: -2px; }
    .hero p { font-size: 15px; }
    .workspace { gap: 12px; }
    .panel { padding: 20px; border-radius: 19px; }
    .mini-grid { grid-template-columns: 1fr; }
  }
```

Code shown in video — timestamp 00:13

```
254   .mini-grid article {
257   }
258   .mini-grid span { color: #53677f; font-size: 10px; }
259   .mini-grid strong { display: block; margin-top: 18px; font: 600 14px "Space Grotesk"; }
260   .mini-grid p { color: #718399; font-size: 12px; line-height: 1.5; margin-bottom: 0; }
261
262   footer { text-align: center; color: #4f637b; font-size: 11px; padding: 10px 20px 35px; }
263
264   @media (max-width: 850px) {
265     .workspace { grid-template-columns: 1fr; }
266     .panel { min-height: auto; }
267     .mini-grid { grid-template-columns: 1fr 1fr; }
268   }
269   @media (max-width: 560px) {
270     .topbar { padding: 18px; }
271     .status { display: none; }
272     .shell { padding: 10px 15px 35px; }
273     .hero { padding: 40px 0 28px; }
274     .hero h1 { letter-spacing: -2px; }
275     .hero p { font-size: 15px; }
276     .workspace { gap: 12px; }
277     .panel { padding: 20px; border-radius: 19px; }
278     .mini-grid { grid-template-columns: 1fr; }
279   }
280
```

Code shown in video — timestamp 00:15

```
196
197  .empty-state, .loading {
198    min-height: 360px; display: grid; place-content: center; text-align: center;
199    color: var(--muted); padding: 30px;
200  }
201  .empty-icon {
202    margin: auto auto 18px; width: 58px; height: 58px; display: grid; place-items: center;
203    border: 1px solid rgba(99,230,190,.2); border-radius: 18px;
204    color: var(--accent); background: rgba(99,230,190,.06); font-size: 24px;
205  }
206  .empty-state h3 { color: #dce6f2; margin: 0 0 7px; font: 600 18px "Space Grotesk"; }
207  .empty-state p { max-width: 330px; margin: 0; line-height: 1.6; font-size: 13px; }
208
209  .loader {
210    width: 32px; height: 32px; margin: auto; border-radius: 50%;
211    border: 3px solid rgba(255,255,255,.1); border-top-color: var(--accent);
212    animation: spin .8s linear infinite;
213  }
214  .loading p { font-size: 13px; }
215  @keyframes spin { to { transform: rotate(360deg); } }
216
217  .result { color: #cbd7e7; line-height: 1.75; font-size: 14px; }
218  .result h3 { color: #f1f6fd; font-family: "Space Grotesk"; }
219  .result strong { color: white; }
220  .hidden { display: none !important; }
221
222  .quiz-card {
223    padding: 18px; margin-bottom: 14px; border: 1px solid var(--line);
224    border-radius: 16px; background: rgba(255,255,255,.02);
225  }
226  .quiz-number { color: var(--accent); font-size: 10px; letter-spacing: 1.5px; font-weight: 700; }
227  .quiz-card h3 { font-size: 15px; margin: 9px 0 13px; line-height: 1.5; }
```

Code shown in video — timestamp 00:16

```
textarea::placeholder { color: #53677f; }

.examples { display: flex; flex-wrap: wrap; gap: 7px; margin: 13px 0 18px; }
.examples button {
  border: 0; color: #8295ac; background: transparent; padding: 0;
  font-size: 12px; cursor: pointer;
}
.examples button:hover { color: var(--accent); }

.primary-btn {
  width: 100%;
  border: 0;
  border-radius: 14px;
  padding: 15px 17px;
  color: #07111f;
  background: var(--accent);
  font: 700 14px "DM Sans";
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 9px;
  transition: .2s;
}
.primary-btn:hover { transform: translateY(-1px); filter: brightness(1.04); }
.primary-btn:disabled { opacity: .55; cursor: wait; }
.arrow { margin-left: auto; font-size: 18px; }

.icon-btn {
  border: 1px solid var(--line); color: var(--muted); background: transparent;
  border-radius: 10px; padding: 8px 10px; cursor: pointer;
}
```

Code shown in video — timestamp 00:18

```
121   .feature-chip:hover, .feature-chip.active {
123     background: var(--accent);
124     border-color: var(--accent);
125   }
126
127   .workspace {
128     display: grid;
129     grid-template-columns: 1fr 1fr;
130     gap: 18px;
131   }
132   .panel {
133     min-height: 520px;
134     border: 1px solid var(--line);
135     background: var(--panel);
136     backdrop-filter: blur(18px);
137     border-radius: 25px;
138     padding: 27px;
139     box-shadow: var(--shadow);
140   }
141   .panel-heading {
142     display: flex;
143     align-items: flex-start;
144     justify-content: space-between;
145     gap: 12px;
146   }
147   .panel h2 { margin: 7px 0 25px; font: 600 23px "Space Grotesk"; }
148   .shortcut { font-size: 11px; color: #718399; }
149   label { color: #b8c7da; font-size: 13px; display: block; margin-bottom: 9px; }
150
151   textarea {
152     width: 100%;
153     min-height: 220px;
```

Code shown in video — timestamp 00:19

```
.brand-mark {
  display: grid;
  place-items: center;
  border: 1px solid rgba(99,230,190,.4);
  border-radius: 12px;
  color: var(--accent);
  background: rgba(99,230,190,.08);
}

.status {
  color: var(--muted);
  font-size: 13px;
  display: flex;
  gap: 8px;
  align-items: center;
}
.status-dot {
  width: 7px; height: 7px; border-radius: 50%;
  background: var(--accent);
  box-shadow: 0 0 14px var(--accent);
}

.shell { max-width: 1180px; margin: auto; padding: 30px 24px 50px; }

.hero { max-width: 800px; padding: 60px 0 38px; }
.eyebrow, .panel-kicker {
  color: var(--accent);
  font-size: 11px;
  letter-spacing: 2px;
  font-weight: 700;
}
```

Code shown in video — timestamp 00:21

```
}

.background-orb {
  position: fixed;
  width: 320px;
  height: 320px;
  border-radius: 50%;
  filter: blur(80px);
  opacity: .13;
  pointer-events: none;
}
.orb-one { background: #63e6be; top: 25%; left: -150px; }
.orb-two { background: #6ea8fe; bottom: -130px; right: -100px; }

.topbar {
  max-width: 1180px;
  margin: auto;
  padding: 26px 24px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.brand {
  color: white;
  text-decoration: none;
  display: flex;
  align-items: center;
  gap: 11px;
  font: 700 22px "Space Grotesk";
}
.brand > span:last-child span { color: var(--accent); }
```

Code shown in video — timestamp 00:22

```css
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Space+Grotesk:wgh

:root {
  --bg: #07111f;
  --panel: rgba(15, 28, 47, .82);
  --panel-2: #0d1b2f;
  --line: rgba(164, 190, 220, .14);
  --text: #f3f7ff;
  --muted: #9eafc4;
  --accent: #63e6be;
  --accent-2: #6ea8fe;
  --danger: #ff8e9e;
  --shadow: 0 24px 70px rgba(0,0,0,.25);
}

* { box-sizing: border-box; }

body {
  margin: 0;
  min-height: 100vh;
  background:
    radial-gradient(circle at 15% 15%, rgba(99,230,190,.09), transparent 28%),
    radial-gradient(circle at 90% 20%, rgba(110,168,254,.10), transparent 28%),
    var(--bg);
  color: var(--text);
  font-family: "DM Sans", sans-serif;
}

.background-orb {
  position: fixed;
  width: 320px;
  height: 320px;
```

Code shown in video — timestamp 00:24

```
:root {
  --shadow: 0 24px 70px rgba(0,0,0,.25);
}

* { box-sizing: border-box; }

body {
  margin: 0;
  min-height: 100vh;
  background:
    radial-gradient(circle at 15% 15%, rgba(99,230,190,.09), transparent 28%),
    radial-gradient(circle at 90% 20%, rgba(110,168,254,.10), transparent 28%),
    var(--bg);
  color: var(--text);
  font-family: "DM Sans", sans-serif;
}

.background-orb {
  position: fixed;
  width: 320px;
  height: 320px;
  border-radius: 50%;
  filter: blur(80px);
  opacity: .13;
  pointer-events: none;
}
.orb-one { background: #63e6be; top: 25%; left: -150px; }
.orb-two { background: #6ea8fe; bottom: -130px; right: -100px; }

.topbar {
  max-width: 1180px;
```

Code shown in video — timestamp 00:25

```
}

.background-orb {
  position: fixed;
  width: 320px;
  height: 320px;
  border-radius: 50%;
  filter: blur(80px);
  opacity: .13;
  pointer-events: none;
}
.orb-one { background: #63e6be; top: 25%; left: -150px; }
.orb-two { background: #6ea8fe; bottom: -130px; right: -100px; }

.topbar {
  max-width: 1180px;
  margin: auto;
  padding: 26px 24px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.brand {
  color: white;
  text-decoration: none;
  display: flex;
  align-items: center;
  gap: 11px;
  font: 700 22px "Space Grotesk";
}
.brand > span:last-child span { color: var(--accent); }
```

Code shown in video — timestamp 00:27

```
.topbar {
  max-width: 1180px;
  margin: auto;
  padding: 26px 24px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.brand {
  color: white;
  text-decoration: none;
  display: flex;
  align-items: center;
  gap: 11px;
  font: 700 22px "Space Grotesk";
}
.brand > span:last-child span { color: var(--accent); }

.brand-mark {
  width: 38px;
  height: 38px;
  display: grid;
  place-items: center;
  border: 1px solid rgba(99,230,190,.4);
  border-radius: 12px;
  color: var(--accent);
  background: rgba(99,230,190,.08);
}

.status {
```

Code shown in video — timestamp 00:28

```
59
60  .brand-mark {
61    width: 38px;
62    height: 38px;
63    display: grid;
64    place-items: center;
65    border: 1px solid rgba(99,230,190,.4);
66    border-radius: 12px;
67    color: var(--accent);
68    background: rgba(99,230,190,.08);
69  }
70
71  .status {
72    color: var(--muted);
73    font-size: 13px;
74    display: flex;
75    gap: 8px;
76    align-items: center;
77  }
78  .status-dot {
79    width: 7px; height: 7px; border-radius: 50%;
80    background: var(--accent);
81    box-shadow: 0 0 14px var(--accent);
82  }
83
84  .shell { max-width: 1180px; margin: auto; padding: 30px 24px 50px; }
85
86  .hero { max-width: 800px; padding: 60px 0 38px; }
87  .eyebrow, .panel-kicker {
88    color: var(--accent);
89    font-size: 11px;
90    letter-spacing: 2px;
```

Code shown in video — timestamp 00:30

```
71   .status {
73     font-size: 13px;
74     display: flex;
75     gap: 8px;
76     align-items: center;
77   }
78   .status-dot {
79     width: 7px; height: 7px; border-radius: 50%;
80     background: var(--accent);
81     box-shadow: 0 0 14px var(--accent);
82   }
83
84   .shell { max-width: 1180px; margin: auto; padding: 30px 24px 50px; }
85
86   .hero { max-width: 800px; padding: 60px 0 38px; }
87   .eyebrow, .panel-kicker {
88     color: var(--accent);
89     font-size: 11px;
90     letter-spacing: 2px;
91     font-weight: 700;
92   }
93   .hero h1 {
94     font: 700 clamp(44px, 6vw, 76px)/1.02 "Space Grotesk";
95     letter-spacing: -3px;
96     margin: 14px 0 20px;
97   }
98   .hero h1 span {
99     background: linear-gradient(90deg, var(--accent), #8cb8ff);
100    -webkit-background-clip: text;
101    background-clip: text;
102    color: transparent;
103  }
```

Code shown in video — timestamp 00:31

```
87   .eyebrow, .panel-kicker {
92   }
93   .hero h1 {
94     font: 700 clamp(44px, 6vw, 76px)/1.02 "Space Grotesk";
95     letter-spacing: -3px;
96     margin: 14px 0 20px;
97   }
98   .hero h1 span {
99     background: linear-gradient(90deg, var(--accent), #8cb8ff);
100    -webkit-background-clip: text;
101    background-clip: text;
102    color: transparent;
103  }
104  .hero p {
105    color: var(--muted);
106    font-size: 18px;
107    line-height: 1.7;
108    max-width: 680px;
109  }
110
111  .feature-row { display: flex; flex-wrap: wrap; gap: 9px; margin-top: 28px; }
112  .feature-chip {
113    border: 1px solid var(--line);
114    color: #c9d5e6;
115    background: rgba(255,255,255,.025);
116    border-radius: 999px;
117    padding: 10px 15px;
118    cursor: pointer;
119    transition: .2s ease;
120  }
121  .feature-chip:hover, .feature-chip.active {
```

Code shown in video — timestamp 00:33

```
.feature-row { display: flex; flex-wrap: wrap; gap: 9px; margin-top: 28px; }
.feature-chip {
  border: 1px solid var(--line);
  color: #c9d5e6;
  background: rgba(255,255,255,.025);
  border-radius: 999px;
  padding: 10px 15px;
  cursor: pointer;
  transition: .2s ease;
}
.feature-chip:hover, .feature-chip.active {
  color: #07111f;
  background: var(--accent);
  border-color: var(--accent);
}

.workspace {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 18px;
}
.panel {
  min-height: 520px;
  border: 1px solid var(--line);
  background: var(--panel);
  backdrop-filter: blur(18px);
  border-radius: 25px;
  padding: 27px;
  box-shadow: var(--shadow);
}
.panel-heading {
  display: flex;
```

Code shown in video — timestamp 00:34

```
127   .workspace {
131   }
132   .panel {
133     min-height: 520px;
134     border: 1px solid var(--line);
135     background: var(--panel);
136     backdrop-filter: blur(18px);
137     border-radius: 25px;
138     padding: 27px;
139     box-shadow: var(--shadow);
140   }
141   .panel-heading {
142     display: flex;
143     align-items: flex-start;
144     justify-content: space-between;
145     gap: 12px;
146   }
147   .panel h2 { margin: 7px 0 25px; font: 600 23px "Space Grotesk"; }
148   .shortcut { font-size: 11px; color: #718399; }
149   label { color: #b8c7da; font-size: 13px; display: block; margin-bottom: 9px; }
150
151   textarea {
152     width: 100%;
153     min-height: 220px;
154     resize: vertical;
155     background: #091628;
156     color: var(--text);
157     border: 1px solid var(--line);
158     border-radius: 16px;
159     padding: 17px;
160     font: 15px/1.6 "DM Sans";
161     outline: none;
```

Code shown in video — timestamp 00:36

```
151  textarea {
155    background: #091628;
156    color: var(--text);
157    border: 1px solid var(--line);
158    border-radius: 16px;
159    padding: 17px;
160    font: 15px/1.6 "DM Sans";
161    outline: none;
162  }
163  textarea:focus { border-color: rgba(99,230,190,.55); box-shadow: 0 0 0 4px rgba(99,230,190,.06); }
164  textarea::placeholder { color: #53677f; }
165
166  .examples { display: flex; flex-wrap: wrap; gap: 7px; margin: 13px 0 18px; }
167  .examples button {
168    border: 0; color: #8295ac; background: transparent; padding: 0;
169    font-size: 12px; cursor: pointer;
170  }
171  .examples button:hover { color: var(--accent); }
172
173  .primary-btn {
174    width: 100%;
175    border: 0;
176    border-radius: 14px;
177    padding: 15px 17px;
178    color: #07111f;
179    background: var(--accent);
180    font: 700 14px "DM Sans";
181    cursor: pointer;
182    display: flex;
183    align-items: center;
184    justify-content: center;
185    gap: 9px;
```

Code shown in video — timestamp 00:37

```
173  .primary-btn {
178    color: #071111;
179    background: var(--accent);
180    font: 700 14px "DM Sans";
181    cursor: pointer;
182    display: flex;
183    align-items: center;
184    justify-content: center;
185    gap: 9px;
186    transition: .2s;
187  }
188  .primary-btn:hover { transform: translateY(-1px); filter: brightness(1.04); }
189  .primary-btn:disabled { opacity: .55; cursor: wait; }
190  .arrow { margin-left: auto; font-size: 18px; }
191
192  .icon-btn {
193    border: 1px solid var(--line); color: var(--muted); background: transparent;
194    border-radius: 10px; padding: 8px 10px; cursor: pointer;
195  }
196
197  .empty-state, .loading {
198    min-height: 360px; display: grid; place-content: center; text-align: center;
199    color: var(--muted); padding: 30px;
200  }
201  .empty-icon {
202    margin: auto auto 18px; width: 58px; height: 58px; display: grid; place-items: center;
203    border: 1px solid rgba(99,230,190,.2); border-radius: 18px;
204    color: var(--accent); background: rgba(99,230,190,.06); font-size: 24px;
205  }
206  .empty-state h3 { color: #dce6f2; margin: 0 0 7px; font: 600 18px "Space Grotesk"; }
207  .empty-state p { max-width: 330px; margin: 0; line-height: 1.6; font-size: 13px; }
208
```

Code shown in video — timestamp 00:39

```
196
197  .empty-state, .loading {
198    min-height: 360px; display: grid; place-content: center; text-align: center;
199    color: var(--muted); padding: 30px;
200  }
201  .empty-icon {
202    margin: auto auto 18px; width: 58px; height: 58px; display: grid; place-items: center;
203    border: 1px solid rgba(99,230,190,.2); border-radius: 18px;
204    color: var(--accent); background: rgba(99,230,190,.06); font-size: 24px;
205  }
206  .empty-state h3 { color: #dce6f2; margin: 0 0 7px; font: 600 18px "Space Grotesk"; }
207  .empty-state p { max-width: 330px; margin: 0; line-height: 1.6; font-size: 13px; }
208
209  .loader {
210    width: 32px; height: 32px; margin: auto; border-radius: 50%;
211    border: 3px solid rgba(255,255,255,.1); border-top-color: var(--accent);
212    animation: spin .8s linear infinite;
213  }
214  .loading p { font-size: 13px; }
215  @keyframes spin { to { transform: rotate(360deg); } }
216
217  .result { color: #cbd7e7; line-height: 1.75; font-size: 14px; }
218  .result h3 { color: #f1f6fd; font-family: "Space Grotesk"; }
219  .result strong { color: white; }
220  .hidden { display: none !important; }
221
222  .quiz-card {
223    padding: 18px; margin-bottom: 14px; border: 1px solid var(--line);
224    border-radius: 16px; background: rgba(255,255,255,.02);
225  }
226  .quiz-number { color: var(--accent); font-size: 10px; letter-spacing: 1.5px; font-weight: 700; }
227  .quiz-card h3 { font-size: 15px; margin: 9px 0 13px; line-height: 1.5; }
```

Code shown in video — timestamp 00:40

```
209   .loader {
213   }
214   .loading p { font-size: 13px; }
215   @keyframes spin { to { transform: rotate(360deg); } }
216
217   .result { color: #cbd7e7; line-height: 1.75; font-size: 14px; }
218   .result h3 { color: #f1f6fd; font-family: "Space Grotesk"; }
219   .result strong { color: white; }
220   .hidden { display: none !important; }
221
222   .quiz-card {
223     padding: 18px; margin-bottom: 14px; border: 1px solid var(--line);
224     border-radius: 16px; background: rgba(255,255,255,.02);
225   }
226   .quiz-number { color: var(--accent); font-size: 10px; letter-spacing: 1.5px; font-weight: 700; }
227   .quiz-card h3 { font-size: 15px; margin: 9px 0 13px; line-height: 1.5; }
228   .options { display: grid; gap: 8px; }
229   .option {
230     border: 1px solid var(--line); background: #0a1728; color: #b9c7d8;
231     border-radius: 10px; padding: 10px; text-align: left; cursor: pointer;
232     display: flex; gap: 10px; align-items: center;
233   }
234   .option span {
235     width: 24px; height: 24px; display: grid; place-items: center;
236     border-radius: 7px; background: rgba(255,255,255,.06); font-size: 11px;
237   }
238   .option:hover { border-color: rgba(99,230,190,.45); }
239   .option.correct { border-color: var(--accent); color: var(--accent); }
240   .option.wrong { border-color: var(--danger); color: var(--danger); }
241   .option:disabled { cursor: default; }
242   .quiz-feedback { margin-top: 9px; font-size: 12px; color: var(--muted); }
```

Code shown in video — timestamp 00:42

```
.quiz-card h3 { font-size: 15px; margin: 9px 0 13px; line-height: 1.5; }
.options { display: grid; gap: 8px; }
.option {
  border: 1px solid var(--line); background: #0a1728; color: #b9c7d8;
  border-radius: 10px; padding: 10px; text-align: left; cursor: pointer;
  display: flex; gap: 10px; align-items: center;
}
.option span {
  width: 24px; height: 24px; display: grid; place-items: center;
  border-radius: 7px; background: rgba(255,255,255,.06); font-size: 11px;
}
.option:hover { border-color: rgba(99,230,190,.45); }
.option.correct { border-color: var(--accent); color: var(--accent); }
.option.wrong { border-color: var(--danger); color: var(--danger); }
.option:disabled { cursor: default; }
.quiz-feedback { margin-top: 9px; font-size: 12px; color: var(--muted); }

.error-box {
  padding: 17px; border: 1px solid rgba(255,142,158,.25);
  background: rgba(255,142,158,.06); border-radius: 13px; color: var(--danger);
}
.error-box p { margin-bottom: 0; color: #c9a7ad; }

.mini-grid {
  display: grid; grid-template-columns: repeat(4, 1fr);
  gap: 10px; margin-top: 18px;
}
.mini-grid article {
  padding: 19px; border: 1px solid var(--line); border-radius: 17px;
  background: rgba(255,255,255,.018);
}
.mini-grid span { color: #53677f; font-size: 10px; }
```

Code shown in video — timestamp 00:43

```
.error-box p { margin-bottom: 0; color: #c9a7ad; }

.mini-grid {
  display: grid; grid-template-columns: repeat(4, 1fr);
  gap: 10px; margin-top: 18px;
}
.mini-grid article {
  padding: 19px; border: 1px solid var(--line); border-radius: 17px;
  background: rgba(255,255,255,.018);
}
.mini-grid span { color: #53677f; font-size: 10px; }
.mini-grid strong { display: block; margin-top: 18px; font: 600 14px "Space Grotesk"; }
.mini-grid p { color: #718399; font-size: 12px; line-height: 1.5; margin-bottom: 0; }

footer { text-align: center; color: #4f637b; font-size: 11px; padding: 10px 20px 35px; }

@media (max-width: 850px) {
  .workspace { grid-template-columns: 1fr; }
  .panel { min-height: auto; }
  .mini-grid { grid-template-columns: 1fr 1fr; }
}
@media (max-width: 560px) {
  .topbar { padding: 18px; }
  .status { display: none; }
  .shell { padding: 10px 15px 35px; }
  .hero { padding: 40px 0 28px; }
  .hero h1 { letter-spacing: -2px; }
  .hero p { font-size: 15px; }
  .workspace { gap: 12px; }
  .panel { padding: 20px; border-radius: 19px; }
  .mini-grid { grid-template-columns: 1fr; }
}
```

Code shown in video — timestamp 00:45

```
259   .mini-grid strong { display: block; margin-top: 18px; font: 600 14px "Space Grotesk"; }
260   .mini-grid p { color: #718399; font-size: 12px; line-height: 1.5; margin-bottom: 0; }
261
262   footer { text-align: center; color: #4f637b; font-size: 11px; padding: 10px 20px 35px; }
263
264   @media (max-width: 850px) {
265     .workspace { grid-template-columns: 1fr; }
266     .panel { min-height: auto; }
267     .mini-grid { grid-template-columns: 1fr 1fr; }
268   }
269   @media (max-width: 560px) {
270     .topbar { padding: 18px; }
271     .status { display: none; }
272     .shell { padding: 10px 15px 35px; }
EduGenie_complete_project\EduGenie\tests
x 0 28px; }
274     .hero h1 { letter-spacing: -2px; }
275     .hero p { font-size: 15px; }
276     .workspace { gap: 12px; }
277     .panel { padding: 20px; border-radius: 19px; }
278     .mini-grid { grid-template-columns: 1fr; }
279   }
280
```

Code shown in video — timestamp 00:46

```
<html lang="en">
<body>
<script>
function formatText(text) {
    .replace(/^### (.*)$/gm, "<h3>$1</h3>")
    .replace(/^## (.*)$/gm, "<h3>$1</h3>")
    .replace(/\n/g, "<br>");
}

function escapeHtml(value) {
  return value.replace(/[&<>"']/g, c => ({
    "&":"&amp;", "<":"&lt;", ">":"&gt;", '"':"&quot;", "'":"&#039;"
  }[c]));
}

function escapeAttr(value) {
  return escapeHtml(String(value));
}
</script>
</body>
</html>
```

Tab to Jump Line 3

Code shown in video — timestamp 00:48

```
  2  <html lang="en">
 10  <body>
102  <script>
179  async function runTask() {
202      const data = await response.json();
203      if (!response.ok) throw new Error(data.detail || "Something went wrong.");
204
205      renderResult(data);
206    } catch (error) {
207      $("result").innerHTML = `<div class="error-box"><strong>Could not complete the request.</strong><p>${es
208      $("result").classList.remove("hidden");
209    } finally {
210      $("loadingState").classList.add("hidden");
211      $("submitBtn").disabled = false;
212    }
213  }
214
215  function renderResult(data) {
216    if (data.quiz) {
217      $("result").innerHTML = data.quiz.map((q, i) => `
218        <div class="quiz-card">
219          <div class="quiz-number">QUESTION ${i + 1}</div>
220          <h3>${escapeHtml(q.question)}</h3>
221          <div class="options">
222            ${q.options.map((o, idx) => `
223              <button class="option" data-answer="${escapeAttr(q.answer)}" data-value="${escapeAttr(o)}">
224                <span>${String.fromCharCode(65 + idx)}</span>${escapeHtml(o)}
225              </button>
226            `).join("")}
227          </div>
228          <div class="quiz-feedback"></div>
229        </div>
```

Tab to Jump Line 3

Code shown in video — timestamp 00:49

```
<html lang="en">
<body>
<script>
const modes = {
  },
  learn: {
    title: "Build a learning path",
    label: "What do you want to master?",
    placeholder: "Example: Learn SQL from beginner to advanced.",
    button: "Create learning path",
    icon: "🗺️"
  }
};

let currentTask = "qa";
const $ = (id) => document.getElementById(id);

document.querySelectorAll(".feature-chip").forEach(btn => {
  btn.addEventListener("click", () => {
    currentTask = btn.dataset.task;
    document.querySelectorAll(".feature-chip").forEach(x => x.classList.remove("active"));
    btn.classList.add("active");
    const mode = modes[currentTask];
    $("taskTitle").textContent = mode.title;
    $("inputLabel").textContent = mode.label;
    $("inputText").placeholder = mode.placeholder;
    $("btnText").textContent = mode.button;
    $("btnIcon").textContent = mode.icon;
    $("inputText").focus();
  });
});
```

Code shown in video — timestamp 00:51

```
  2   <html lang="en">
 10   <body>
 22     <main class="shell">
 96     </main>
 97
 98     <footer>
 99       EduGenie · Built with FastAPI + Google Gemini
100     </footer>
101
102   <script>
103   const modes = {
104     qa: {
105       title: "Ask EduGenie",
106       label: "What would you like to learn?",
107       placeholder: "Example: Why is the sky blue?",
108       button: "Ask EduGenie",
109       icon: "✦"
110     },
111     explain: {
112       title: "Explain a concept",
113       label: "Which concept should I simplify?",
114       placeholder: "Example: Explain photosynthesis for a class 8 student.",
115       button: "Explain concept",
116       icon: "🧠"
117     },
118     quiz: {
119       title: "Generate a quiz",
120       label: "Paste a topic, lesson, or passage",
121       placeholder: "Paste your study material here…",
122       button: "Generate quiz",
123       icon: "📝"
124     },
```

Code shown in video — timestamp 00:52

```
<html lang="en">
<body>
  <main class="shell">
    <section class="hero">
      <p>
        Ask questions, simplify concepts, create quizzes, summarize notes,
        and build a step-by-step learning path with EduGenie.
      </p>

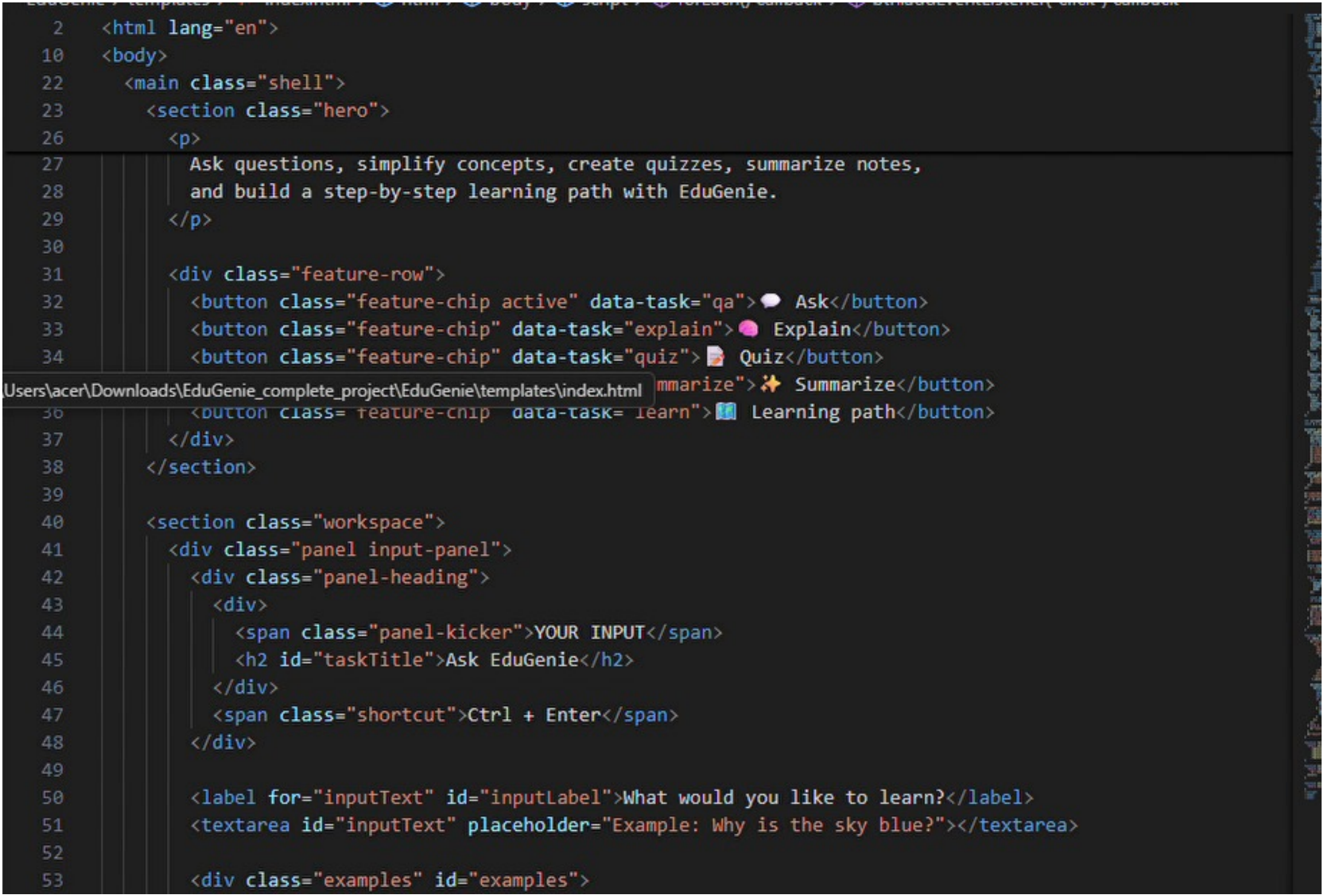
      <div class="feature-row">
        <button class="feature-chip active" data-task="qa">💬 Ask</button>
        <button class="feature-chip" data-task="explain">🧠 Explain</button>
        <button class="feature-chip" data-task="quiz">📝 Quiz</button>
        <button class="feature-chip" data-task="summarize">✨ Summarize</button>
        <button class="feature-chip" data-task="learn">🗺️ Learning path</button>
      </div>
    </section>

    <section class="workspace">
      <div class="panel input-panel">
        <div class="panel-heading">
          <div>
            <span class="panel-kicker">YOUR INPUT</span>
            <h2 id="taskTitle">Ask EduGenie</h2>
          </div>
          <span class="shortcut">Ctrl + Enter</span>
        </div>

        <label for="inputText" id="inputLabel">What would you like to learn?</label>
        <textarea id="inputText" placeholder="Example: Why is the sky blue?"></textarea>

        <div class="examples" id="examples">
```

Code shown in video — timestamp 00:54



```
 2   <html lang="en">
10   <body>
22     <main class="shell">
23       <section class="hero">
26         <p>
27           Ask questions, simplify concepts, create quizzes, summarize notes,
28           and build a step-by-step learning path with EduGenie.
29         </p>
30
31         <div class="feature-row">
32           <button class="feature-chip active" data-task="qa">💬 Ask</button>
33           <button class="feature-chip" data-task="explain">🧠 Explain</button>
34           <button class="feature-chip" data-task="quiz">📝 Quiz</button>
             C:\Users\acer\Downloads\EduGenie_complete_project\EduGenie\templates\index.html
             mmarize">✨ Summarize</button>
36           <button class="feature-chip" data-task="learn">🗺️ Learning path</button>
37         </div>
38       </section>
39
40       <section class="workspace">
41         <div class="panel input-panel">
42           <div class="panel-heading">
43             <div>
44               <span class="panel-kicker">YOUR INPUT</span>
45               <h2 id="taskTitle">Ask EduGenie</h2>
46             </div>
47             <span class="shortcut">Ctrl + Enter</span>
48           </div>
49
50           <label for="inputText" id="inputLabel">What would you like to learn?</label>
51           <textarea id="inputText" placeholder="Example: Why is the sky blue?"></textarea>
52
53           <div class="examples" id="examples">
```

Code shown in video — timestamp 00:55

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="description" content="EduGenie - your AI-powered learning assistant" />
  <title>EduGenie — Learn Smarter</title>
  <link rel="stylesheet" href="{{ url_for('static', path='style.css') }}" />
</head>
<body>
  <div class="background-orb orb-one"></div>
  <div class="background-orb orb-two"></div>

  <header class="topbar">
    <a class="brand" href="/">
      <span class="brand-mark">E</span>
      <span>Edu<span>Genie</span></span>
    </a>
    <div class="status"><span class="status-dot"></span> AI learning assistant</div>
  </header>

  <main class="shell">
    <section class="hero">
      <div class="eyebrow">PERSONALIZED • SIMPLE • SMART</div>
      <h1>Turn difficult topics into <span>clear learning.</span></h1>
      <p>
        Ask questions, simplify concepts, create quizzes, summarize notes,
        and build a step-by-step learning path with EduGenie.
      </p>

      <div class="feature-row">
        <button class="feature-chip active" data-task="qa">💬 Ask</button>
```

Code shown in video — timestamp 00:57

```
<html lang="en">
<body>
  <header class="topbar">
  </header>

  <main class="shell">
    <section class="hero">
      <div class="eyebrow">PERSONALIZED • SIMPLE • SMART</div>
      <h1>Turn difficult topics into <span>clear learning.</span></h1>
      <p>
        Ask questions, simplify concepts, create quizzes, summarize notes,
        and build a step-by-step learning path with EduGenie.
      </p>

      <div class="feature-row">
        <button class="feature-chip active" data-task="qa">💬 Ask</button>
        <button class="feature-chip" data-task="explain">🧠 Explain</button>
        <button class="feature-chip" data-task="quiz">📝 Quiz</button>
        <button class="feature-chip" data-task="summarize">✨ Summarize</button>
        <button class="feature-chip" data-task="learn">🗺️ Learning path</button>
      </div>
    </section>

    <section class="workspace">
      <div class="panel input-panel">
        <div class="panel-heading">
          <div>
            <span class="panel-kicker">YOUR INPUT</span>
            <h2 id="taskTitle">Ask EduGenie</h2>
          </div>
          <span class="shortcut">Ctrl + Enter</span>
        </div>
```

Code shown in video — timestamp 00:58

```
<html lang="en">
<body>
  <main class="shell">
    <section class="hero">
      </p>

      <div class="feature-row">
        <button class="feature-chip active" data-task="qa">💬 Ask</button>
        <button class="feature-chip" data-task="explain">🧠 Explain</button>
        <button class="feature-chip" data-task="quiz">📝 Quiz</button>
        <button class="feature-chip" data-task="summarize">✨ Summarize</button>
        <button class="feature-chip" data-task="learn">🗺️ Learning path</button>
      </div>
    </section>

    <section class="workspace">
      <div class="panel input-panel">
        <div class="panel-heading">
          <div>
            <span class="panel-kicker">YOUR INPUT</span>
            <h2 id="taskTitle">Ask EduGenie</h2>
          </div>
          <span class="shortcut">Ctrl + Enter</span>
        </div>

        <label for="inputText" id="inputLabel">What would you like to learn?</label>
        <textarea id="inputText" placeholder="Example: Why is the sky blue?"></textarea>

        <div class="examples" id="examples">
          <button>Explain photosynthesis</button>
          <button>What is the Pythagoras theorem?</button>
          <button>How do rivers shape landscapes?</button>
```

Code shown in video — timestamp 01:00

```
<html lang="en">
<body>
  <main class="shell">
    <section class="workspace">
      <div class="panel input-panel">
        <div class="examples" id="examples">
          <button>Explain photosynthesis</button>
          <button>What is the Pythagoras theorem?</button>
          <button>How do rivers shape landscapes?</button>
        </div>

        <button id="submitBtn" class="primary-btn">
          <span id="btnIcon">✦</span>
          <span id="btnText">Ask EduGenie</span>
          <span class="arrow">→</span>
        </button>
      </div>

      <div class="panel result-panel">
        <div class="panel-heading">
          <div>
            <span class="panel-kicker">EDUGENIE</span>
            <h2>Learning result</h2>
          </div>
          <button id="copyBtn" class="icon-btn" title="Copy result">⧉</button>
        </div>

        <div id="emptyState" class="empty-state">
          <div class="empty-icon">✦</div>
          <h3>Your answer will appear here</h3>
          <p>Choose a learning mode, enter your content, and let EduGenie help you learn.</p>
        </div>
```

Code shown in video — timestamp 01:01

```
<html lang="en">
<body>
  <main class="shell">
    <section class="workspace">
      <div class="panel input-panel">
      </div>

      <div class="panel result-panel">
        <div class="panel-heading">
          <div>
            <span class="panel-kicker">EDUGENIE</span>
            <h2>Learning result</h2>
          </div>
          <button id="copyBtn" class="icon-btn" title="Copy result">⧉</button>
        </div>

        <div id="emptyState" class="empty-state">
          <div class="empty-icon">✦</div>
          <h3>Your answer will appear here</h3>
          <p>Choose a learning mode, enter your content, and let EduGenie help you learn.</p>
        </div>

        <div id="loadingState" class="loading hidden">
          <div class="loader"></div>
          <p>Thinking and preparing your learning material…</p>
        </div>

        <div id="result" class="result hidden"></div>
      </div>
    </section>

    <section class="mini-grid">
```

Code shown in video — timestamp 01:03

```
<html lang="en">
<body>
  <main class="shell">
    <section class="workspace">
      <div class="panel result-panel">

        <div id="result" class="result hidden"></div>
      </div>
    </section>

    <section class="mini-grid">
      <article><span>01</span><strong>Ask anything</strong><p>Get concise academic answers.</p></article>
      <article><span>02</span><strong>Understand better</strong><p>Break complex concepts into simple langu
      <article><span>03</span><strong>Practice actively</strong><p>Generate a three-question MCQ quiz.</p><
      <article><span>04</span><strong>Plan your growth</strong><p>Move from beginner to advanced.</p></arti
    </section>
  </main>

  <footer>
    EduGenie · Built with FastAPI + Google Gemini
  </footer>

<script>
const modes = {
  qa: {
    title: "Ask EduGenie",
    label: "What would you like to learn?",
    placeholder: "Example: Why is the sky blue?",
    button: "Ask EduGenie",
    icon: "✦"
  },
  explain: {
```

Code shown in video — timestamp 01:04

```
  <html lang="en">
  <body>
    <main class="shell">
      <section class="mini-grid">
      </section>
    </main>

    <footer>
      EduGenie · Built with FastAPI + Google Gemini
    </footer>

  <script>
  const modes = {
    qa: {
      title: "Ask EduGenie",
      label: "What would you like to learn?",
      placeholder: "Example: Why is the sky blue?",
      button: "Ask EduGenie",
      icon: "✦"
    },
    explain: {
      title: "Explain a concept",
      label: "Which concept should I simplify?",
      placeholder: "Example: Explain photosynthesis for a class 8 student.",
      button: "Explain concept",
      icon: "🧠"
    },
    quiz: {
      title: "Generate a quiz",
      label: "Paste a topic, lesson, or passage",
      placeholder: "Paste your study material here…",
```

Code shown in video — timestamp 01:06

```
<html lang="en">
<body>
<script>
const modes = {
  quiz: {
    button: "Generate quiz",
    icon: "📝"
  },
  summarize: {
    title: "Summarize your notes",
    label: "Paste the educational text",
    placeholder: "Paste a long paragraph or lesson here…",
    button: "Summarize",
    icon: "✨"
  },
  learn: {
    title: "Build a learning path",
    label: "What do you want to master?",
    placeholder: "Example: Learn SQL from beginner to advanced.",
    button: "Create learning path",
    icon: "🗺️"
  }
};

let currentTask = "qa";
const $ = (id) => document.getElementById(id);

document.querySelectorAll(".feature-chip").forEach(btn => {
  btn.addEventListener("click", () => {
    currentTask = btn.dataset.task;
    document.querySelectorAll(".feature-chip").forEach(x => x.classList.remove("active"));
    btn.classList.add("active");
```

Code shown in video — timestamp 01:07

```
  2   <html lang="en">
 10   <body>
102   <script>
144   document.querySelectorAll(".feature-chip").forEach(btn => {
145     btn.addEventListener("click", () => {
148       btn.classList.add("active");
149       const mode = modes[currentTask];
150       $("taskTitle").textContent = mode.title;
151       $("inputLabel").textContent = mode.label;
152       $("inputText").placeholder = mode.placeholder;
153       $("btnText").textContent = mode.button;
154       $("btnIcon").textContent = mode.icon;
155       $("inputText").focus();
156     });
157   });
158
159   document.querySelectorAll(".examples button").forEach(btn => {
160     btn.addEventListener("click", () => {
161       $("inputText").value = btn.textContent;
162       $("inputText").focus();
163     });
164   });
165
166   $("submitBtn").addEventListener("click", runTask);
167   $("inputText").addEventListener("keydown", e => {
168     if (e.ctrlKey && e.key === "Enter") runTask();
169   });
170
171   $("copyBtn").addEventListener("click", async () => {
172     const text = $("result").innerText.trim();
173     if (!text) return;
174     await navigator.clipboard.writeText(text);
```

Code shown in video — timestamp 01:09

```
  2   <html lang="en">
 10   <body>
102   <script>
171   $("copyBtn").addEventListener("click", async () => {
174     await navigator.clipboard.writeText(text);
175     $("copyBtn").textContent = "✓";
176     setTimeout(() => $("copyBtn").textContent = "⧉", 1000);
177   });
178
179   async function runTask() {
180     const text = $("inputText").value.trim();
181     if (!text) {
182       $("inputText").focus();
183       return;
184     }
185
186     $("emptyState").classList.add("hidden");
187     $("result").classList.add("hidden");
188     $("loadingState").classList.remove("hidden");
189     $("submitBtn").disabled = true;
190
191     const endpoint = currentTask === "learn"
192       ? "/learn/recommendations"
193       : `/${currentTask}`;
194
195     try {
196       const response = await fetch(endpoint, {
197         method: "POST",
198         headers: {"Content-Type": "application/json"},
199         body: JSON.stringify({text})
200       });
```

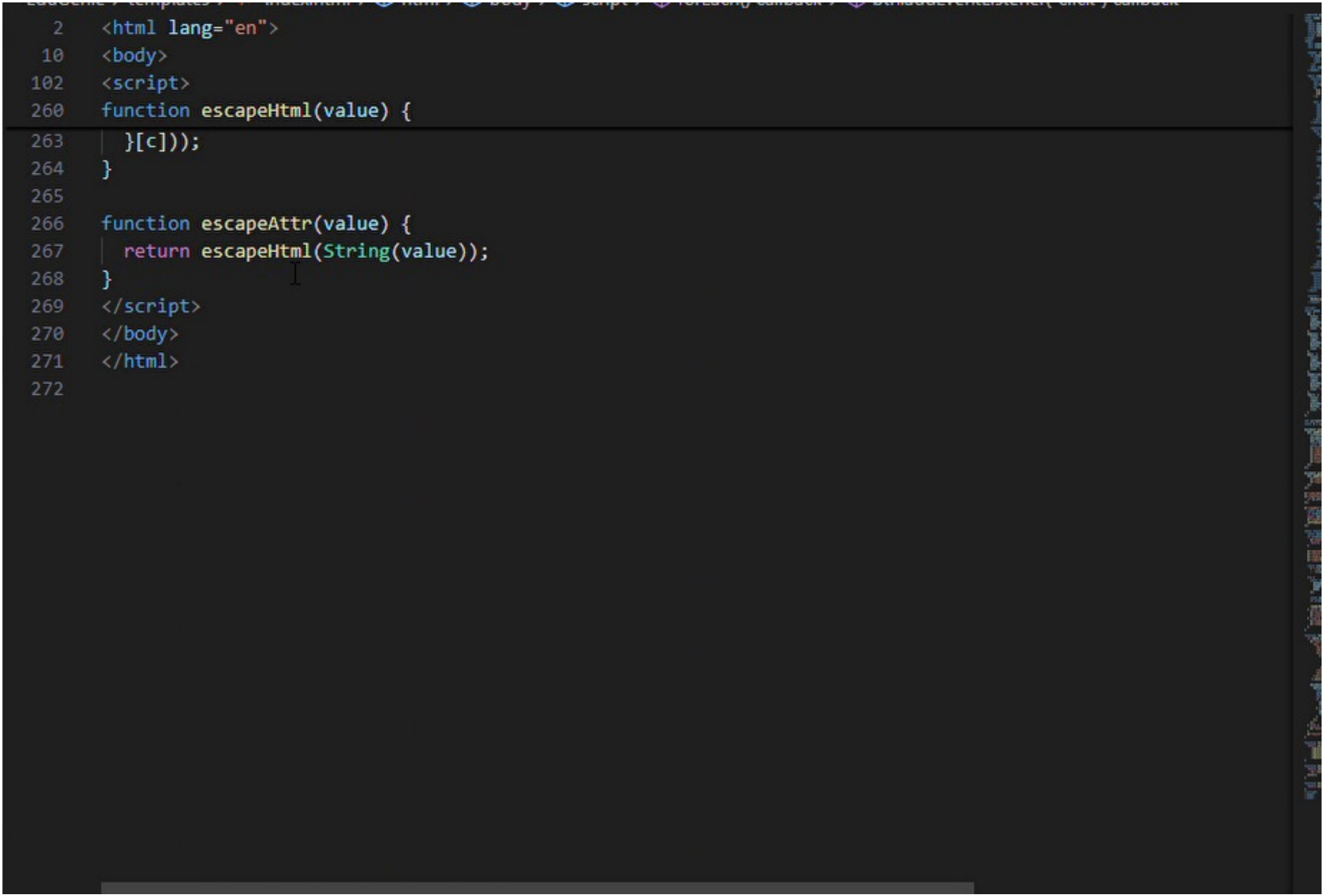
Code shown in video — timestamp 01:10

```
  2   <html lang="en">
 10   <body>
102   <script>
179   async function runTask() {
200       });
201
202       const data = await response.json();
203       if (!response.ok) throw new Error(data.detail || "Something went wrong.");
204
205       renderResult(data);
206     } catch (error) {
207       $("result").innerHTML = `<div class="error-box"><strong>Could not complete the request.</strong><p>${es
208       $("result").classList.remove("hidden");
209     } finally {
210       $("loadingState").classList.add("hidden");
211       $("submitBtn").disabled = false;
212     }
213   }
214
215   function renderResult(data) {
216     if (data.quiz) {
217       $("result").innerHTML = data.quiz.map((q, i) => `
218         <div class="quiz-card">
219           <div class="quiz-number">QUESTION ${i + 1}</div>
220           <h3>${escapeHtml(q.question)}</h3>
221           <div class="options">
222             ${q.options.map((o, idx) => `
223               <button class="option" data-answer="${escapeAttr(q.answer)}" data-value="${escapeAttr(o)}">
224                 <span>${String.fromCharCode(65 + idx)}</span>${escapeHtml(o)}
225               </button>
226             `).join("")}
227           </div>
```

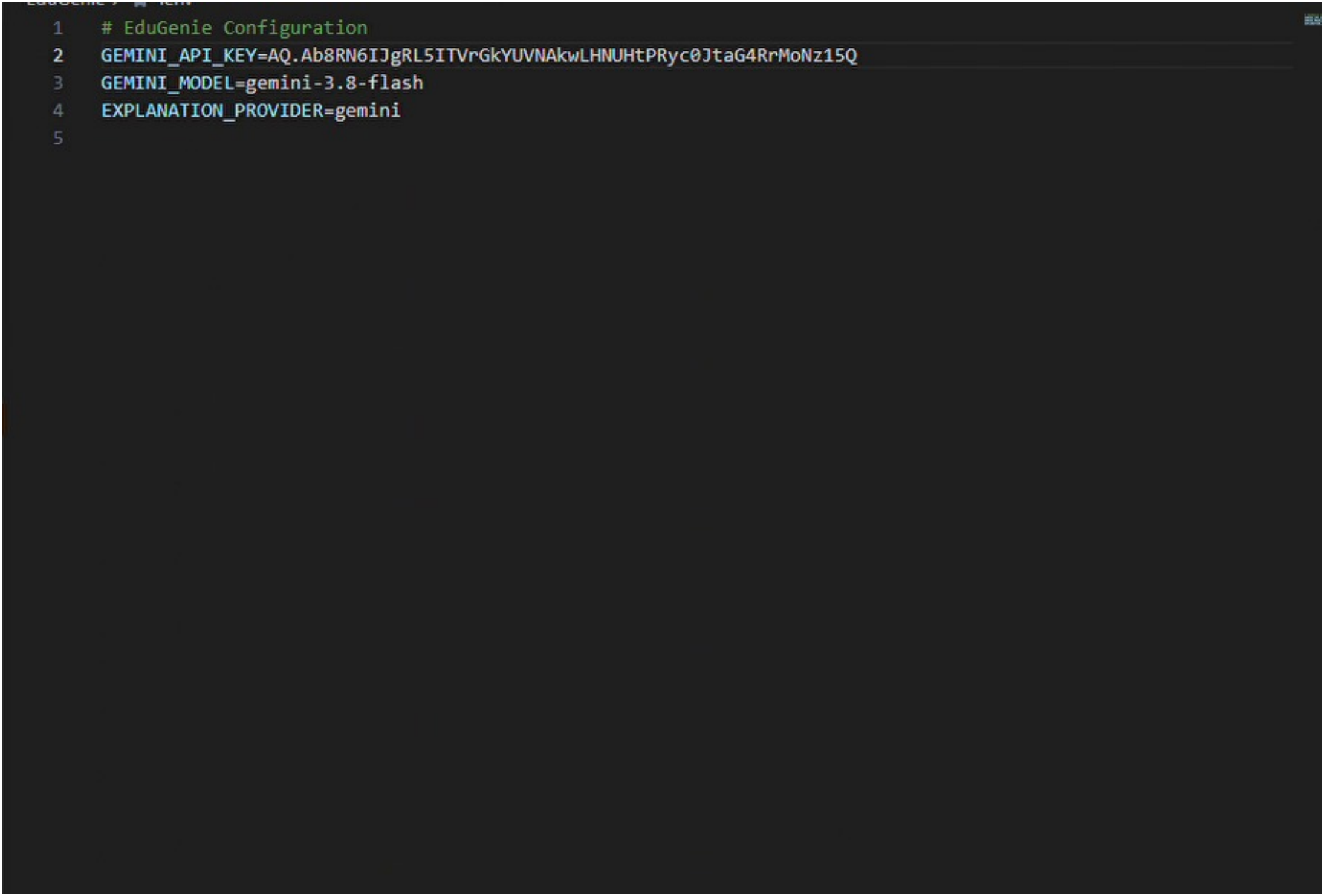
Code shown in video — timestamp 01:12

```
  2   <html lang="en">
 10   <body>
102   <script>
215   function renderResult(data) {
217       $("result").innerHTML = data.quiz.map((q, i) => `
230       `).join("");
231
232       document.querySelectorAll(".option").forEach(option => {
233         option.addEventListener("click", () => {
234           const card = option.closest(".quiz-card");
235           card.querySelectorAll(".option").forEach(x => x.disabled = true);
236           const feedback = card.querySelector(".quiz-feedback");
237           if (option.dataset.value === option.dataset.answer) {
238             option.classList.add("correct");
239             feedback.textContent = "✓ Correct!";
240           } else {
241             option.classList.add("wrong");
242             feedback.textContent = `Correct answer: ${option.dataset.answer}`;
243           }
244         });
245       });
246     } else {
247       $("result").innerHTML = formatText(data.result || "");
248     }
249     $("result").classList.remove("hidden");
250   }
251
252   function formatText(text) {
253     return escapeHtml(text)
254       .replace(/\*\*(.*?)\*\*/g, "<strong>$1</strong>")
255       .replace(/^### (.*)$/gm, "<h3>$1</h3>")
256       .replace(/^## (.*)$/gm, "<h3>$1</h3>")
```

Code shown in video — timestamp 01:13



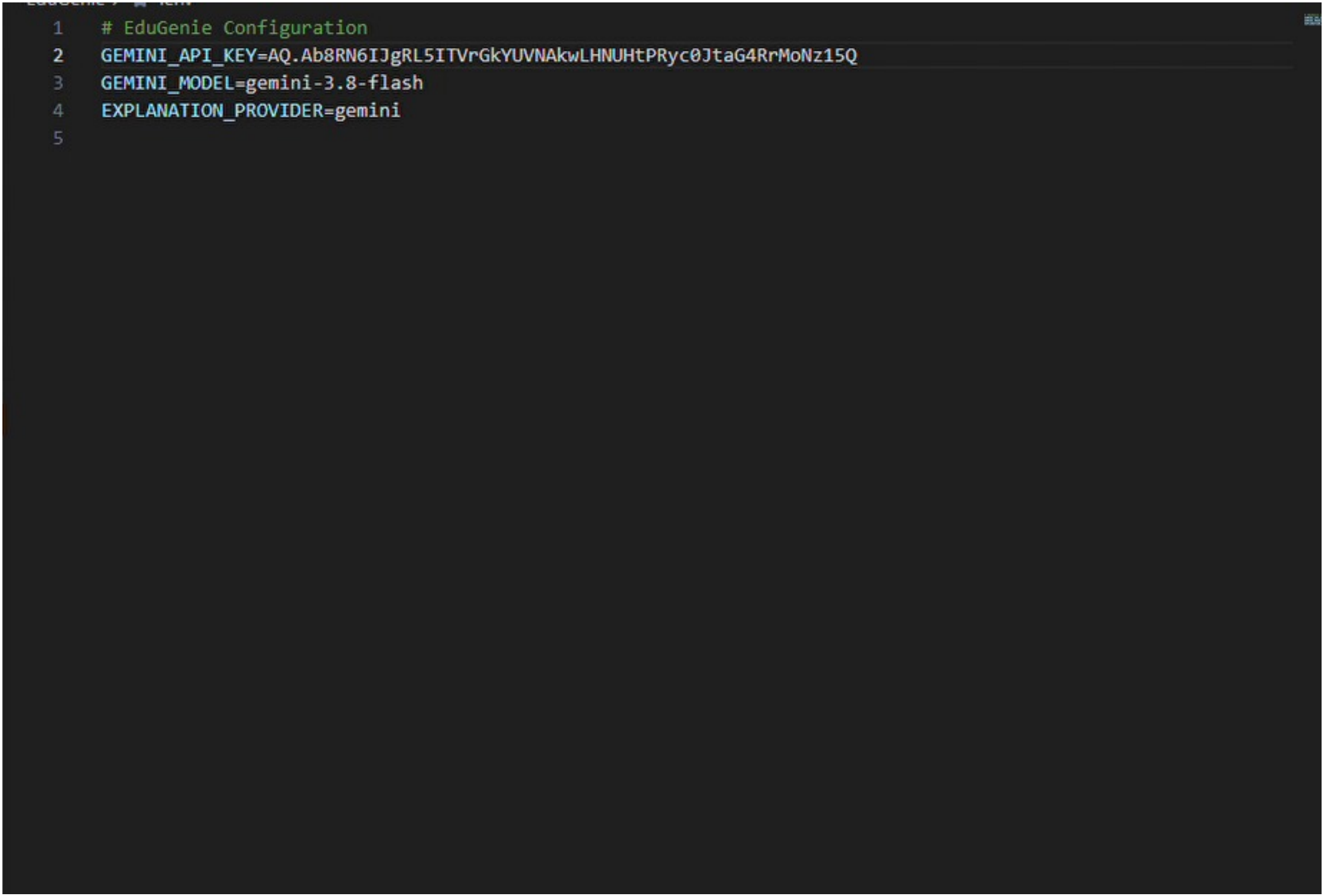
Code shown in video — timestamp 01:15



```
# EduGenie Configuration
GEMINI_API_KEY=AQ.Ab8RN6IJgRL5ITVrGkYUVNAkwLHNUHtPRyc0JtaG4RrMoNz15Q
GEMINI_MODEL=gemini-3.8-flash
EXPLANATION_PROVIDER=gemini

```

Code shown in video — timestamp 01:16



```
# EduGenie Configuration
GEMINI_API_KEY=AQ.Ab8RN6IJgRL5ITVrGkYUVNAkwLHNUHtPRyc0JtaG4RrMoNz15Q
GEMINI_MODEL=gemini-3.8-flash
EXPLANATION_PROVIDER=gemini

```

Code shown in video — timestamp 01:18

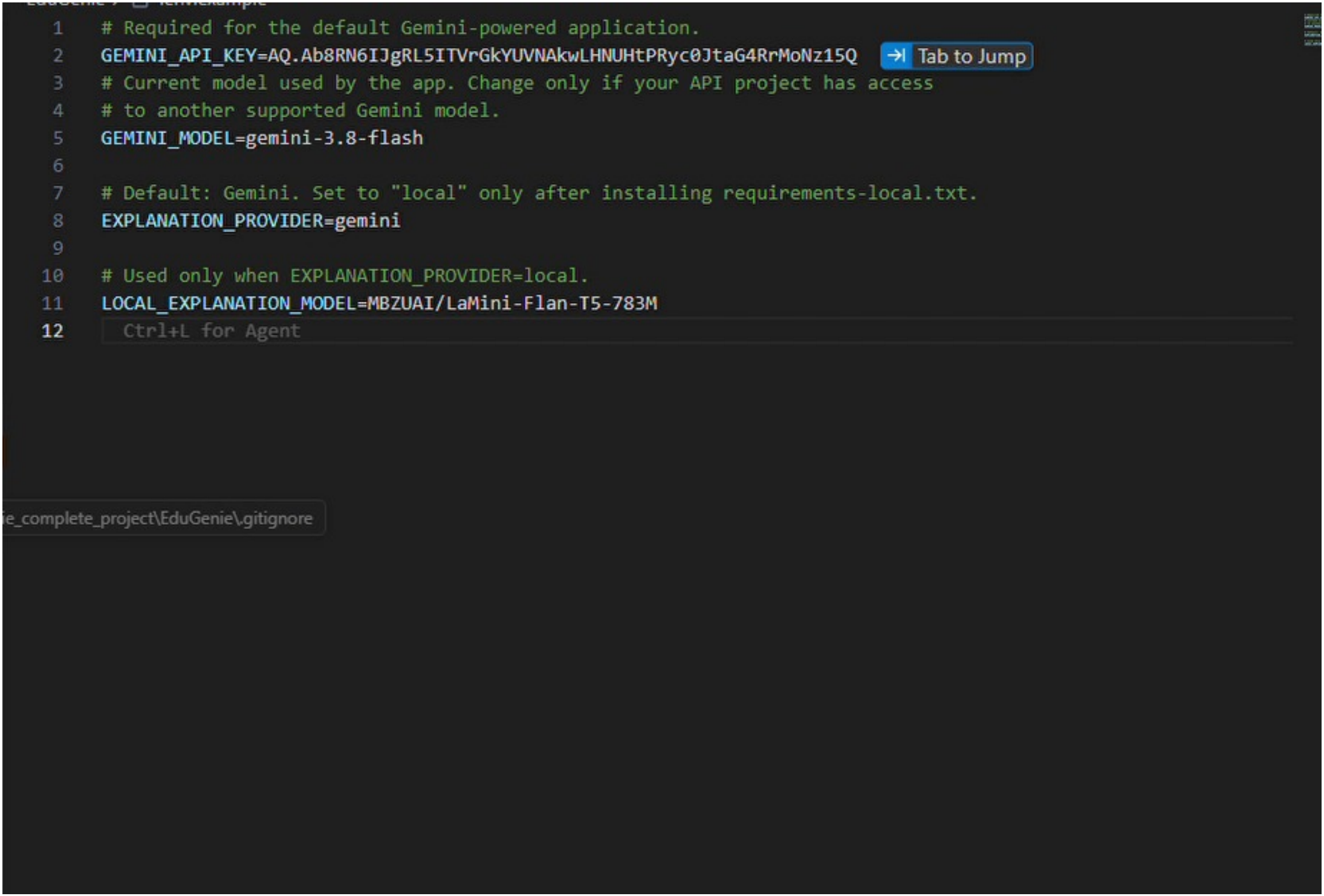
```
# to another supported Gemini model.
GEMINI_MODEL=gemini-3.8-flash

# Default: Gemini. Set to "local" only after installing requirements-local.txt.
EXPLANATION_PROVIDER=gemini

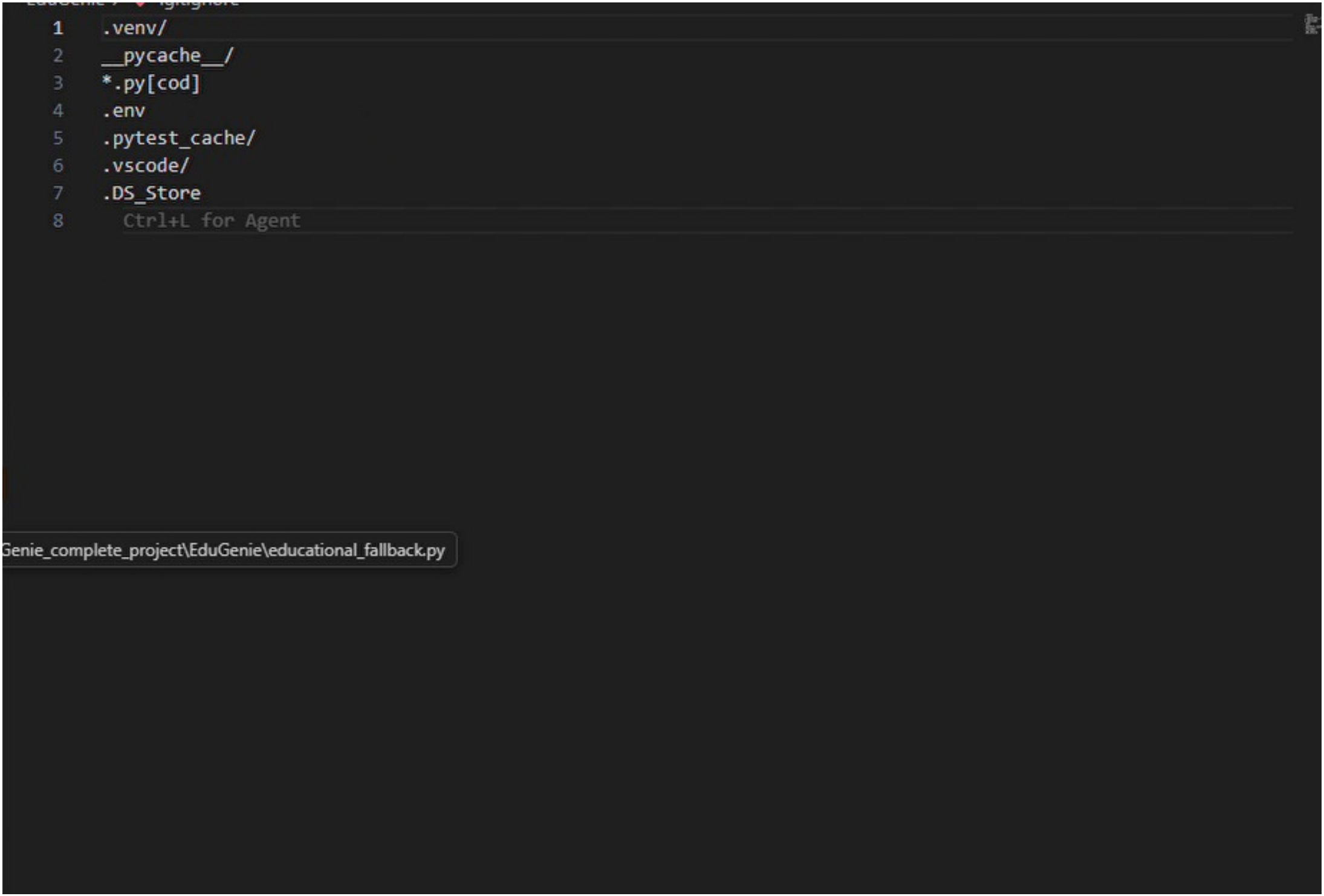
# Used only when EXPLANATION_PROVIDER=local.
LOCAL_EXPLANATION_MODEL=MBZUAI/LaMini-Flan-T5-783M
```

Tab to Jump Line 2

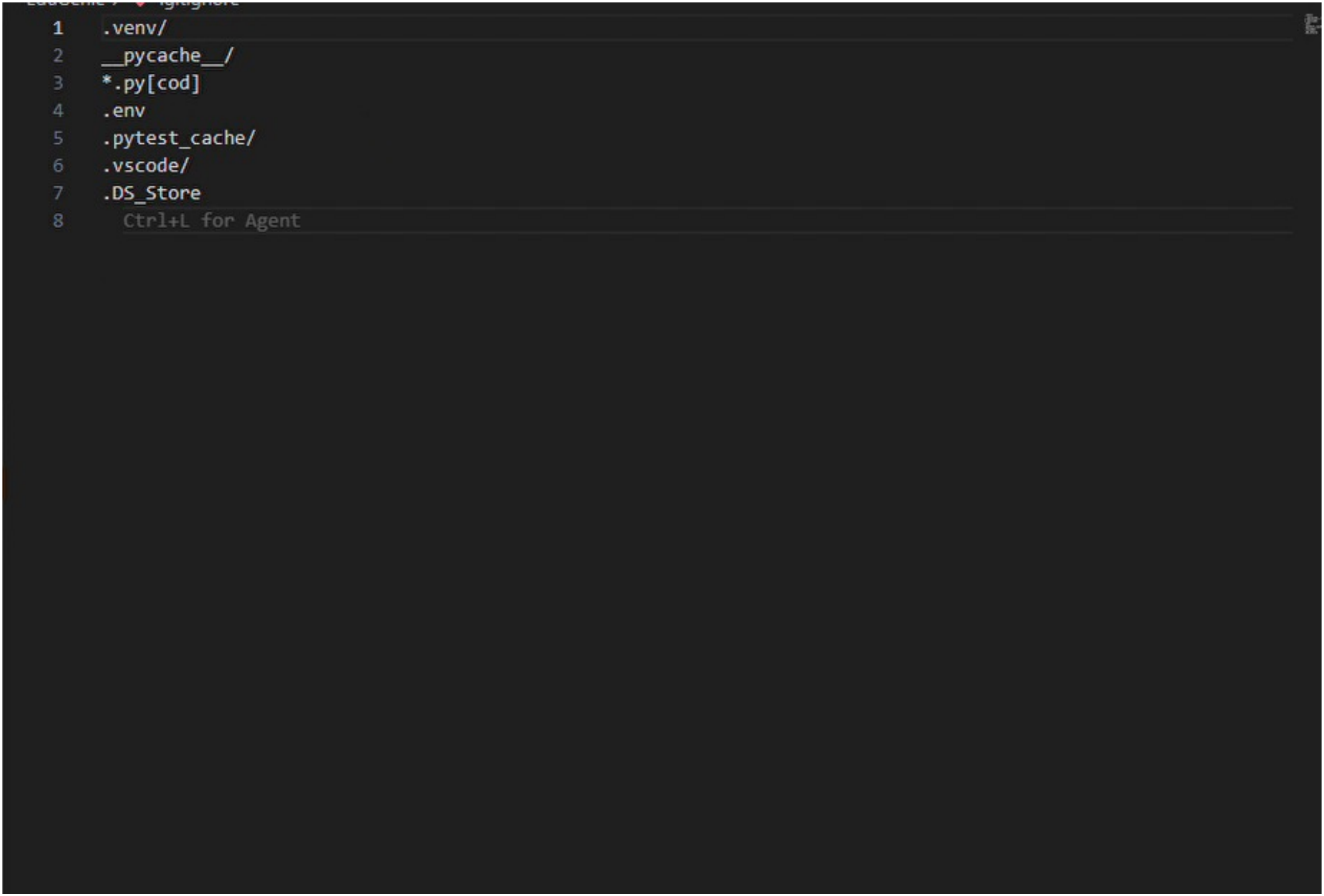
Code shown in video — timestamp 01:19



Code shown in video — timestamp 01:21



Code shown in video — timestamp 01:22



Code shown in video — timestamp 01:24

```python
def fallback_quiz(passage: str) -> str:
    questions = [
        {
            "question": "Approximately how long does it take for Earth to complete one full revolution
            "options": ["365.25 days (1 year)", "24 hours (1 day)", "28 days (1 month)", "10 years (1 d
            "answer": "365.25 days (1 year)"
        },
        {
            "question": "What astronomical body revolves directly around the Earth?",
            "options": ["The Moon", "The Sun", "Venus", "Saturn"],
            "answer": "The Moon"
        }
    ]

    return json.dumps({"questions": questions})


def fallback_summary(passage: str) -> str:
    words = [w for w in passage.split() if len(w) > 3]
    subject = words[0] if words else "Educational Topic"

    return (
        f"### Quick Revision Summary\n\n"
        f"**Core Essence:**\n"
        f"The provided text outlines essential principles regarding **{subject}**, highlighting structural
        f"**Key Takeaways & Facts:**\n"
        f"• **Foundational Principle:** Key concepts operate sequentially, establishing clear cause-and-eff
        f"• **Critical Components:** Essential variables work synchronously to ensure structural and functi
        f"• **Practical Implications:** Understanding these core facts allows quick recall, accurate analys
        f"**Exam Tip:** Focus on the definitions, relationships between primary variables, and real-world e
    )

```

Code shown in video — timestamp 01:25

```
 98    def fallback_quiz(passage: str) -> str:
102        questions = [
111                "answer": f"To provide a systematic mechanism for {subject} processes"
112            },
113            {
114                "question": f"Which of the following is a key characteristic of {subject}?",
115                "options": [
116                    "It operates according to verifiable principles and structured stages",
117                    "It violates basic conservation and scientific laws",
118                    "It can only be applied in non-reproducible conditions",
119                    "It produces identical outputs regardless of initial inputs"
120                ],
121                "answer": "It operates according to verifiable principles and structured stages"
122            },
123            {
124                "question": f"In practical study, how is {subject} most effectively analyzed?",
125                "options": [
126                    "By breaking it down into inputs, transformation mechanisms, and measurable outcomes",
127                    "By ignoring baseline data and focusing solely on extreme edge cases",
128                    "By assuming outcomes occur without any causal relationships",
129                    "By replacing empirical observation with unverified assumptions"
130                ],
131                "answer": "By breaking it down into inputs, transformation mechanisms, and measurable outcomes"
132            }
133        ]
134
135        p_lower = passage.lower()
136        if "photosynthesis" in p_lower:
137            questions = [
138                {
139                    "question": "What are the primary reactants required for photosynthesis?",
140                    "options": ["Carbon dioxide, water, and sunlight", "Oxygen, glucose, and nitrogen", "Carbon
```

Code shown in video — timestamp 01:27

```
12  def fallback_qa(question: str) -> str:
55          return (
56              "### How Rivers Shape Landscapes\n"
57              "Rivers shape the Earth's surface continuously through three linked geological processes: **ero
58              "**Key Mechanisms:**\n"
59              "1. **Erosion (Upper Course):** Fast-flowing mountain streams use hydraulic action and abrasion
60              "2. **Transportation (Middle Course):** As the gradient softens, the river carries eroded sedim
61              "3. **Deposition (Lower Course):** When velocity drops near lakes or the ocean, sediment settle
62              "**Key Takeaway:** Rivers act as dynamic sculpting agents that carve valleys in highlands and d
63          )
64
65      return (
66          f"### Academic Answer: {question.rstrip('?')}\n\n"
67          f"**Core Concept:**\n"
68          f"Regarding **{subject}**, the core principle revolves around understanding its fundamental mechani
69          f"**Key Principles & Explanatory Points:**\n"
70          f"• **Definition & Function:** {subject.capitalize()} functions as a primary construct in its field
71          f"• **Operational Mechanism:** It progresses systematically from foundational conditions to measura
72          f"• **Practical Application:** In real-world contexts, understanding {subject} enables accurate pro
73          f"**Key Takeaway:**\n"
74          f"Mastery of {subject} requires connecting basic definitions to practical, observable applications
75      )
76
77
78  def fallback_explain(topic: str) -> str:
79      clean_topic = _extract_topic_or_subject(topic)
80
81      return (
82          f"### Explaining: {clean_topic.capitalize()}\n\n"
83          f"**What is it in simple terms?**\n"
84          f"{clean_topic.capitalize()} is a fundamental educational concept that describes how key elements i
85          f"**How it works step-by-step:**\n"
```

Code shown in video — timestamp 01:28

```python
def fallback_qa(question: str) -> str:
    subject = _extract_topic_or_subject(question)
    q_lower = question.lower()

    if "photosynthesis" in q_lower:
        return (
            "### Photosynthesis Overview\n"
            "Photosynthesis is the biological process by which green plants, algae, and certain bacteria co
            "**Chemical Equation:**\n"
            "6CO₂ + 6H₂O + Light Energy → C₆H₁₂O₆ + 6O₂\n\n"
            "**Key Stages:**\n"
            "1. **Light-Dependent Reactions (Thylakoids):** Sunlight is absorbed by chlorophyll pigments, s
            "2. **Light-Independent Reactions / Calvin Cycle (Stroma):** Carbon dioxide (CO₂) is fixed usin
nloads\EduGenie_complete_project\EduGenie\templates\index.html
            thesis produces oxygen essential for aerobic life and forms the prim
        )

    if "pythagor" in q_lower or "pythagorean" in q_lower:
        return (
            "### The Pythagorean Theorem\n"
            "The Pythagorean theorem states that in any right-angled triangle, the square of the hypotenuse
            "**Mathematical Formula:**\n"
            "**a² + b² = c²**\n"
            "*(where `c` is the length of the hypotenuse, and `a` and `b` are the lengths of the perpendicu
            "**Step-by-Step Example:**\n"
            "- Suppose a triangle has legs `a = 3 cm` and `b = 4 cm`.\n"
            "- `c² = 3² + 4² = 9 + 16 = 25`\n"
            "- `c = √25 = 5 cm`.\n\n"
            "**Key Takeaway:** This fundamental geometric principle is essential in navigation, constructio
        )

    if "sky" in q_lower and "blue" in q_lower:
```

Code shown in video — timestamp 01:30

```python
import json
import re
from typing import List, Dict, Any


def _extract_topic_or_subject(text
    cleaned = re.sub(r'(?i)^(what
    cleaned = cleaned.rstrip('?.!
    return cleaned if cleaned else


def fallback_qa(question: str) -> str:
    subject = _extract_topic_or_subject(question)
    q_lower = question.lower()

    if "photosynthesis" in q_lower:
        return (
            "### Photosynthesis Overview\n"
            "Photosynthesis is the biological process by which green plants, algae, and certain bacteria co
            "**Chemical Equation:**\n"
            "6CO₂ + 6H₂O + Light Energy → C₆H₁₂O₆ + 6O₂\n\n"
            "**Key Stages:**\n"
            "1. **Light-Dependent Reactions (Thylakoids):** Sunlight is absorbed by chlorophyll pigments, s
            "2. **Light-Independent Reactions / Calvin Cycle (Stroma):** Carbon dioxide (CO₂) is fixed usin
            "**Key Takeaway:** Photosynthesis produces oxygen essential for aerobic life and forms the prim
        )

    if "pythagor" in q_lower or "pythagorean" in q_lower:
        return (
            "### The Pythagorean Theorem\n"
            "The Pythagorean theorem states that in any right-angled triangle, the square of the hypotenuse
            "**Mathematical Formula:**\n"
```

```
@overload
def __new__(object: object = '') -> str: ...
def __new__(
    object  : Buffer,
    encoding: str = 'utf-8',
    errors  : str = 'strict'
) -> str: ...
```

str(object='') -> str
str(bytes_or_buffer[, encoding[, errors]]) -> str

Create a new string object from the given object. If encoding or errors is specified, then the object must expose a data buffer

lo|can you explain|

Code shown in video — timestamp 01:31

```python
import json
import re
from typing import List, Dict, Any


def _extract_topic_or_subject(text: str) -> str:
    cleaned = re.sub(r'(?i)^(what is|what are|explain|how does|why is|tell me about|how do|can you explain|
    cleaned = cleaned.rstrip('?.! ')
    return cleaned if cleaned else text.strip()

(function) _extract_topic_or_subject: def _extract_topic_or_subject(text: str) -> str: ...
Go to str

def fallback_q
    subject = _extract_topic_or_subject(question)
    q_lower = question.lower()

    if "photosynthesis" in q_lower:
        return (
            "### Photosynthesis Overview\n"
            "Photosynthesis is the biological process by which green plants, algae, and certain bacteria co
            "**Chemical Equation:**\n"
            "6CO₂ + 6H₂O + Light Energy → C₆H₁₂O₆ + 6O₂\n\n"
            "**Key Stages:**\n"
            "1. **Light-Dependent Reactions (Thylakoids):** Sunlight is absorbed by chlorophyll pigments, s
            "2. **Light-Independent Reactions / Calvin Cycle (Stroma):** Carbon dioxide (CO₂) is fixed usin
            "**Key Takeaway:** Photosynthesis produces oxygen essential for aerobic life and forms the prim
        )

    if "pythagor" in q_lower or "pythagorean" in q_lower:
        return (
            "### The Pythagorean Theorem\n"
            "The Pythagorean theorem states that in any right-angled triangle, the square of the hypotenuse
            "**Mathematical Formula:**\n"
```

Code shown in video — timestamp 01:33

```python
    return cleaned if cleaned else text.strip()


def fallback_qa(question: str) -> str:
    subject = _extract_topic_or_subject(question)
    q_lower = question.lower()

    if "photosynthesis" in q_lower:
        return (
            "### Photosynthesis Overview\n"
            "Photosynthesis is the biological process by which green plants, algae, and certain bacteria co
            "**Chemical Equation:**\n"
            "6CO₂ + 6H₂O + Light Energy → C₆H₁₂O₆ + 6O₂\n\n"
            "**Key Stages:**\n"
            "1. **Light-Dependent Reactions (Thylakoids):** Sunlight is absorbed by chlorophyll pigments, s
            "2. **Light-Independent Reactions / Calvin Cycle (Stroma):** Carbon dioxide (CO₂) is fixed usin
            "**Key Takeaway:** Photosynthesis produces oxygen essential for aerobic life and forms the prim
        )

    if "pythagor" in q_lower or "pythagorean" in q_lower:
        return (
            "### The Pythagorean Theorem\n"
            "The Pythagorean theorem states that in any right-angled triangle, the square of the hypotenuse
            "**Mathematical Formula:**\n"
            "**a² + b² = c²**\n"
            "*(where `c` is the length of the hypotenuse, and `a` and `b` are the lengths of the perpendicu
            "**Step-by-Step Example:**\n"
            "- Suppose a triangle has legs `a = 3 cm` and `b = 4 cm`.\n"
            "- `c² = 3² + 4² = 9 + 16 = 25`\n"
            "- `c = √25 = 5 cm`.\n\n"
            "**Key Takeaway:** This fundamental geometric principle is essential in navigation, constructio
        )
```

Code shown in video — timestamp 01:34

```
12  def fallback_qa(question: str) -> str:
23              "1. **Light-Dependent Reactions (Thylakoids):** Sunlight is absorbed by chlorophyll pigments, s
24              "2. **Light-Independent Reactions / Calvin Cycle (Stroma):** Carbon dioxide (CO₂) is fixed usin
25              "**Key Takeaway:** Photosynthesis produces oxygen essential for aerobic life and forms the prim
26          )
27
28      if "pythagor" in q_lower or "pythagorean" in q_lower:
29          return (
30              "### The Pythagorean Theorem\n"
31              "The Pythagorean theorem states that in any right-angled triangle, the square of the hypotenuse
32              "**Mathematical Formula:**\n"
33              "**a² + b² = c²**\n"
34              "*(where `c` is the length of the hypotenuse, and `a` and `b` are the lengths of the perpendicu
35              "**Step-by-Step Example:**\n"
36              "- Suppose a triangle has legs `a = 3 cm` and `b = 4 cm`.\n"
37              "- `c² = 3² + 4² = 9 + 16 = 25`\n"
38              "- `c = √25 = 5 cm`.\n\n"
39              "**Key Takeaway:** This fundamental geometric principle is essential in navigation, constructio
40          )
41
42      if "sky" in q_lower and "blue" in q_lower:
43          return (
44              "### Why Is the Sky Blue?\n"
45              "The sky appears blue because of an optical phenomenon called **Rayleigh scattering**.\n\n"
46              "**How It Works:**\n"
47              "1. Sunlight reaching Earth's atmosphere is composed of all colors of the visible spectrum (whi
48              "2. Earth's atmosphere is densely packed with tiny gas molecules (mostly nitrogen and oxygen).\
49              "3. Shorter wavelengths of light (blue and violet) scatter in all directions much more strongly
50              "4. Although violet light is scattered even more than blue light, our human eyes are far more s
51              "**Key Takeaway:** Rayleigh scattering disperses short-wavelength blue light across the sky hem
52          )
53
```

```
(variable) q_lower: str
Type source
• Narrowed by condition at 28:22: "pythagor" not in q_lower
Go to str
```

Code shown in video — timestamp 01:36

```python
def fallback_qa(question: str) -> str:

    if "sky" in q_lower and "blue" in q_lower:
        return (
            "### Why Is the Sky Blue?\n"
            "The sky appears blue because of an optical phenomenon called **Rayleigh scattering**.\n\n"
            "**How It Works:**\n"
            "1. Sunlight reaching Earth's atmosphere is composed of all colors of the visible spectrum (whi
            "2. Earth's atmosphere is densely packed with tiny gas molecules (mostly nitrogen and oxygen).\
            "3. Shorter wavelengths of light (blue and violet) scatter in all directions much more strongly
            "4. Although violet light is scattered even more than blue light, our human eyes are far more s
            "**Key Takeaway:** Rayleigh scattering disperses short-wavelength blue light across the sky hem
        )

    if "river" in q_lower and ("shape" in q_lower or "landscape" in q_lower):
        return (
            "### How Rivers Shape Landscapes\n"
            "Rivers shape the Earth's surface continuously through three linked geological processes: **ero
            "**Key Mechanisms:**\n"
            "1. **Erosion (Upper Course):** Fast-flowing mountain streams use hydraulic action and abrasion
            "2. **Transportation (Middle Course):** As the gradient softens, the river carries eroded sedim
            "3. **Deposition (Lower Course):** When velocity drops near lakes or the ocean, sediment settle
            "**Key Takeaway:** Rivers act as dynamic sculpting agents that carve valleys in highlands and d
        )

    return (
        f"### Academic Answer: {question.rstrip('?')}\n\n"
        f"**Core Concept:**\n"
        f"Regarding **{subject}**, the core principle revolves around understanding its fundamental mechani
        f"**Key Principles & Explanatory Points:**\n"
        f"• **Definition & Function:** {subject.capitalize()} functions as a primary construct in its field
        f"• **Operational Mechanism:** It progresses systematically from foundational conditions to measura
```

Code shown in video — timestamp 01:37

```
def fallback_qa(question: str) -> str:
        f"**Key Takeaway:**\n"
        f"Mastery of {subject} requires connecting basic definitions to practical, observable applications
    )


def fallback_explain(topic: str) -> str:
    clean_topic = _extract_topic_or_subject(topic)

    return (
        f"### Explaining: {clean_topic.capitalize()}\n\n"
        f"**What is it in simple terms?**\n"
        f"{clean_topic.capitalize()} is a fundamental educational concept that describes how key elements i
        f"**How it works step-by-step:**\n"
        f"1. **Initial State:** Everything begins with the core inputs or baseline principles that set {cle
        f"2. **The Mechanism:** An active process transforms those inputs through specific rules, interacti
        f"3. **The Result:** The system reaches an outcome or balanced state that can be observed, measured
        f"**Simple Everyday Analogy:**\n"
        f"Imagine {clean_topic} like a well-choreographed recipe: each ingredient (component) has an exact
        f"**Real-World Example:**\n"
        f"When practitioners apply {clean_topic} in modern science, technology, or humanities, they rely on
        f"**Summary Note:**\n"
        f"Keep the three stages in mind: inputs, process, and outcome. Once you master this core cycle, adv
    )


def fallback_quiz(passage: str) -> str:
    clean_text = _extract_topic_or_subject(passage)
    subject = clean_text[:40].strip()

    questions = [
        {
```

Code shown in video — timestamp 01:39

```
 98  def fallback_quiz(passage: str) -> str:
102      questions = [
                  f"To eliminate all secondary and peripheral factors completely",
                  f"To restrict observation to purely theoretical calculations"
              ],
              "answer": f"To provide a systematic mechanism for {subject} processes"
          },
          {
              "question": f"Which of the following is a key characteristic of {subject}?",
              "options": [
                  "It operates according to verifiable principles and structured stages",
                  "It violates basic conservation and scientific laws",
                  "It can only be applied in non-reproducible conditions",
                  "It produces identical outputs regardless of initial inputs"
              ],
              "answer": "It operates according to verifiable principles and structured stages"
          },
          {
              "question": f"In practical study, how is {subject} most effectively analyzed?",
              "options": [
                  "By breaking it down into inputs, transformation mechanisms, and measurable outcomes",
                  "By ignoring baseline data and focusing solely on extreme edge cases",
                  "By assuming outcomes occur without any causal relationships",
                  "By replacing empirical observation with unverified assumptions"
              ],
              "answer": "By breaking it down into inputs, transformation mechanisms, and measurable outcomes"
          }
      ]

      p_lower = passage.lower()
      if "photosynthesis" in p_lower:
          questions = [
```

Code shown in video — timestamp 01:40

```python
def fallback_quiz(passage: str) -> str:
        questions = [
            },
            {
                "question": "Which cellular organelle and pigment capture light energy in plant cells?",
                "options": ["Chloroplasts containing chlorophyll", "Mitochondria containing hemoglobin", "R
                "answer": "Chloroplasts containing chlorophyll"
            },
            {
                "question": "What gas is released into the atmosphere as a byproduct of water photolysis du
                "options": ["Oxygen (O₂)", "Carbon dioxide (CO₂)", "Nitrogen (N₂)", "Hydrogen sulfide (H₂S)
                "answer": "Oxygen (O₂)"
            }
        ]
    elif "pythagor" in p_lower:
        questions = [
            {
                "question": "What geometric relationship is expressed by the Pythagorean theorem?",
                "options": ["a² + b² = c² in a right-angled triangle", "a + b = c in an equilateral triangl
                "answer": "a² + b² = c² in a right-angled triangle"
            },
            {
                "question": "If the legs of a right triangle measure 3 cm and 4 cm, what is the length of t
                "options": ["5 cm", "7 cm", "6 cm", "12 cm"],
                "answer": "5 cm"
            },
            {
                "question": "Which side of a right-angled triangle represents the hypotenuse?",
                "options": ["The side directly opposite the 90-degree right angle", "The shortest vertical
                "answer": "The side directly opposite the 90-degree right angle"
            }
        ]
```

Code shown in video — timestamp 01:42

```
 98  def fallback_quiz(passage: str) -> str:
173          questions = [
174              {
175                  "question": "What celestial body does the Earth revolve around?",
176                  "options": ["The Sun", "The Moon", "Mars", "Jupiter"],
177                  "answer": "The Sun"
178              },
179              {
180                  "question": "Approximately how long does it take for Earth to complete one full revolution
181                  "options": ["365.25 days (1 year)", "24 hours (1 day)", "28 days (1 month)", "10 years (1 d
182                  "answer": "365.25 days (1 year)"
183              },
184              {
185                  "question": "What astronomical body revolves directly around the Earth?",
186                  "options": ["The Moon", "The Sun", "Venus", "Saturn"],
187                  "answer": "The Moon"
188              }
189          ]
190
191      return json.dumps({"questions": questions})
192
193
194  def fallback_summary(passage: str) -> str:
195      words = [w for w in passage.split() if len(w) > 3]
196      subject = words[0] if words else "Educational Topic"
197
198      return (
199          f"### Quick Revision Summary\n\n"
200          f"**Core Essence:**\n"
201          f"The provided text outlines essential principles regarding **{subject}**, highlighting structural
202          f"**Key Takeaways & Facts:**\n"
203          f"• **Foundational Principle:** Key concepts operate sequentially, establishing clear cause-and-eff
```

Code shown in video — timestamp 01:43

```
194  def fallback_summary(passage: str) -> str:
205          f"• **Practical Implications:** Understanding these core facts allows quick recall, accurate analys
206          f"**Exam Tip:** Focus on the definitions, relationships between primary variables, and real-world e
207      )
208
209
210  def fallback_learning_path(topic: str) -> str:
211      clean_topic = _extract_topic_or_subject(topic)
212
213      return (
214          f"### Learning Roadmap: {clean_topic.title()}\n\n"
215          f"**Stage 1: Foundations & Core Principles**\n"
216          f"• **Suggested Time:** 1–2 Weeks (4–6 hours/week)\n"
217          f"• **What to Learn:** Core terminology, historical background, foundational theories, and basic bu
218          f"• **Practice Task:** Write a one-page summary explaining the top 3 concepts of {clean_topic} in y
219          f"• **Resource Types:** Introductory YouTube lecture series, foundational textbook chapters, and in
220          f"**Stage 2: Intermediate Application & Problem Solving**\n"
221          f"• **Suggested Time:** 3–4 Weeks (6–8 hours/week)\n"
222          f"• **What to Learn:** Practical methodologies, standard workflows, common tools, and problem-solvi
223          f"• **Practice Task:** Complete 3 guided mini-projects or case studies demonstrating practical end-
224          f"• **Resource Types:** Documentation guides, walkthrough articles, code repositories, and intermed
225          f"**Stage 3: Advanced Specialization & Deep Dive**\n"
226          f"• **Suggested Time:** 2–3 Weeks (8 hours/week)\n"
227          f"• **What to Learn:** Performance optimization, edge cases, advanced architectural patterns, and i
228          f"• **Practice Task:** Analyze an open-source project or complex scenario, refactor an existing sol
229          f"• **Resource Types:** Research papers, expert developer blogs, advanced documentation, and record
230          f"**Stage 4: Capstone Project & Mastery**\n"
231          f"• **Suggested Time:** 2 Weeks (10 hours/week)\n"
232          f"• **What to Learn:** System integration, testing, deployment/presentation, and teaching concepts
233          f"• **Practice Task:** Build and document a comprehensive capstone project around {clean_topic} to
234          f"• **Resource Types:** Community forums, peer review sessions, and open-source contribution guidel
235      )
```

Code shown in video — timestamp 01:45

```
210    def fallback_learning_path(topic: str) -> str:
221            f"• **Suggested Time:** 3–4 Weeks (6–8 hours/week)\n"
222            f"• **What to Learn:** Practical methodologies, standard workflows, common tools, and problem-solvi
223            f"• **Practice Task:** Complete 3 guided mini-projects or case studies demonstrating practical end-
224            f"• **Resource Types:** Documentation guides, walkthrough articles, code repositories, and intermed
225            f"**Stage 3: Advanced Specialization & Deep Dive**\n"
226            f"• **Suggested Time:** 2–3 Weeks (8 hours/week)\n"
227            f"• **What to Learn:** Performance optimization, edge cases, advanced architectural patterns, and i
228            f"• **Practice Task:** Analyze an open-source project or complex scenario, refactor an existing sol
229            f"• **Resource Types:** Research papers, expert developer blogs, advanced documentation, and record
230            f"**Stage 4: Capstone Project & Mastery**\n"
231            f"• **Suggested Time:** 2 Weeks (10 hours/week)\n"
232            f"• **What to Learn:** System integration, testing, deployment/presentation, and teaching concepts
233            f"• **Practice Task:** Build and document a comprehensive capstone project around {clean_topic} to
234            f"• **Resource Types:** Community forums, peer review sessions, and open-source contribution guidel
235        )
236
```

Code shown in video — timestamp 01:46

```python
def _local_explanation(topic: str) -> str:
    """
    Optional local implementation based on the project document's
    LaMini-Flan-T5 approach.

    It is intentionally loaded only when EXPLANATION_PROVIDER=local,
    so normal installation stays lightweight.
    """
    from transformers import pipeline

    model_name = os.getenv("LOCAL_EXPLANATION_MODEL", "MBZUAI/LaMini-Flan-T5-783M")
    generator = pipeline("text2text-generation", model=model_name)
    prompt = (
        "Explain the following educational topic in simple language for a beginner. "
        "Use short paragraphs and one small example.\n\nTopic: " + topic
    )
    result = generator(prompt, max_new_tokens=220, do_sample=False)
    return result[0]["generated_text"].strip()


def explain_concept(topic: str) -> str:
    topic = topic.strip()
    if not topic:
        raise ValueError("Please provide a topic to explain.")

    provider = os.getenv("EXPLANATION_PROVIDER", "gemini").lower()
    if provider == "local":
        return _local_explanation(topic)

    return generate_text(
        "You are EduGenie, a patient educational tutor. "
        "Explain the topic below for a beginner. Be accurate, concise, "
```

Code shown in video — timestamp 01:48

```
4
5
6   def _local_explanation(topic: str) -> str:
7       """
8       Optional local implementation based on the project document's
9       LaMini-Flan-T5 approach.
10
11      It is intentionally loaded only when EXPLANATION_PROVIDER=local,
12      so normal installation stays lightweight.
13      """
14      from transformers import pipeline
15
16      model_name = os.getenv("LOCAL_EXPLANATION_MODEL", "MBZUAI/LaMini-Flan-T5-783M")
17      generator = pipeline("text2text-generation", model=model_name)
18      prompt = (
19          "Explain the following educational topic in simple language for a beginner. "
20          "Use short paragraphs and one small example.\n\nTopic: " + topic
21      )
22      result = generator(prompt, max_new_tokens=220, do_sample=False)
23      return result[0]["generated_text"].strip()
24
25
26  def explain_concept(topic: str) -> str:
27      topic = topic.strip()
28      if not topic:
29          raise ValueError("Please provide a topic to explain.")
30
31      provider = os.getenv("EXPLANATION_PROVIDER", "gemini").lower()
32      if provider == "local":
33          return _local_explanation(topic)
34
35      return generate_text(
```

Code shown in video — timestamp 01:49

```
def _local_explanation(topic: str) -> str:
    prompt = (
        "Explain the following educational topic in simple language for a beginner. "
        "Use short paragraphs and one small example.\n\nTopic: " + topic
    )
    result = generator(prompt, max_new_tokens=220, do_sample=False)
    return result[0]["generated_text"].strip()


def explain_concept(topic: str) -> str:
    topic = topic.strip()
    if not topic:
        raise ValueError("Please provide a topic to explain.")

    provider = os.getenv("EXPLANATION_PROVIDER", "gemini").lower()
    if provider == "local":
        return _local_explanation(topic)

    return generate_text(
        "You are EduGenie, a patient educational tutor. "
        "Explain the topic below for a beginner. Be accurate, concise, "
        "easy to understand, and include a simple example when useful. "
        "Do not assume prior knowledge.\n\n"
        f"Topic: {topic}",
        temperature=0.35,
        max_output_tokens=900,
    )
```

Code shown in video — timestamp 01:51

```
def explain_concept(topic: str) -> str:
        "You are EduGenie, a patient educational tutor. "
        "Explain the topic below for a beginner. Be accurate, concise, "
        "easy to understand, and include a simple example when useful. "
        "Do not assume prior knowledge.\n\n"
        f"Topic: {topic}",
        temperature=0.35,
        max_output_tokens=900,
    )

```

Code shown in video — timestamp 01:52

```python
def generate_text(prompt: str, *, temperature: float = 0.4, max_output_tokens: int = 1200) -> str:

    if "curriculum planner" in prompt_lower or "learning path" in prompt_lower:
        match = re.search(r"topic:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
        topic = match.group(1).strip() if match else prompt
        return fallback_learning_path(topic)

    return fallback_qa(prompt)


def generate_json(prompt: str, *, max_output_tokens: int = 1600) -> str:
    # 1. Attempt Gemini if API key is present
    if _get_api_key():
        try:
            return _call_gemini_json(prompt, max_output_tokens)
        except Exception as exc:
            logger.info("Falling back to local quiz generator: %s", exc)

    # 2. Intelligent quiz fallback
    match = re.search(r"material:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
    material = match.group(1).strip() if match else prompt
    return fallback_quiz(material)

```

Code shown in video — timestamp 01:54

```python
def _call_gemini_json(prompt: str, max_output_tokens: int) -> str:
            client = genai.Client(api_key=api_key)
            response = client.models.generate_content(
                model=model_name,
                contents=prompt,
                config=types.GenerateContentConfig(
                    temperature=0.3,
                    max_output_tokens=max_output_tokens,
                    response_mime_type="application/json",
                ),
            )
            text = getattr(response, "text", None)
            if text and text.strip():
                return text.strip()
        except Exception as exc:
            last_err = exc
            logger.warning("Gemini JSON model %s failed: %s", model_name, exc)
            continue
        finally:
            if client is not None:
                try:
                    if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
                        client._api_client._httpx_client.close()
                except Exception:
                    pass

    raise last_err or RuntimeError("Gemini JSON generation failed.")


def generate_text(prompt: str, *, temperature: float = 0.4, max_output_tokens: int = 1200) -> str:
    # 1. Attempt Gemini if API key is present
```

Code shown in video — timestamp 01:55

```python

class GeminiNotConfiguredError(RuntimeError):
    pass


def _get_api_key() -> str:
    return os.getenv("GEMINI_API_KEY", "").strip()


@lru_cache(maxsize=1)
def get_client():
    api_key = _get_api_key()
    if not api_key:
        raise GeminiNotConfiguredError(
            "GEMINI_API_KEY is not configured. Create a Gemini API key and put it in .env."
        )
    return genai.Client(api_key=api_key)


def _call_gemini_text(prompt: str, temperature: float, max_output_tokens: int) -> str:
    api_key = _get_api_key()
    if not api_key:
        raise GeminiNotConfiguredError("GEMINI_API_KEY not configured")

    candidate_models = [
        MODEL,
        "gemini-3.8-flash",
        "gemini-2.5-flash",
        "gemini-flash-latest",
        "gemini-1.5-flash",
    ]
    seen = set()
```

Code shown in video — timestamp 01:57

```python
import logging
import os
import re
from functools import lru_cache

from dotenv import load_dotenv

load_dotenv()

from google import genai
from google.genai import types

from educational_fallback import (
    fallback_qa,
    fallback_explain,
    fallback_quiz,
    fallback_summary,
    fallback_learning_path,
)

logger = logging.getLogger(__name__)

MODEL = os.getenv("GEMINI_MODEL", "gemini-3.8-flash")


class GeminiNotConfiguredError(RuntimeError):
    pass


def _get_api_key() -> str:
    return os.getenv("GEMINI_API_KEY", "").strip()
```

Code shown in video — timestamp 01:58

```
    fallback_qa,
    fallback_explain,
    fallback_quiz,
    fallback_summary,
    fallback_learning_path,
)

logger = logging.getLogger(__name__)

MODEL = os.getenv("GEMINI_MODEL", "gemini-3.8-flash")


class GeminiNotConfiguredError(RuntimeError):
    pass


def _get_api_key() -> str:
    return os.getenv("GEMINI_API_KEY", "").strip()


@lru_cache(maxsize=1)
def get_client():
    api_key = _get_api_key()
    if not api_key:
        raise GeminiNotConfiguredError(
            "GEMINI_API_KEY is not configured. Create a Gemini API key and put it in .env."
        )
    return genai.Client(api_key=api_key)


def _call_gemini_text(prompt: str, temperature: float, max_output_tokens: int) -> str:
    api_key = _get_api_key()
```

Code shown in video — timestamp 02:00

```python

@lru_cache(maxsize=1)
def get_client():
    api_key = _get_api_key()
    if not api_key:
        raise GeminiNotConfiguredError(
            "GEMINI_API_KEY is not configured. Create a Gemini API key and put it in .env."
        )
    return genai.Client(api_key=api_key)


def _call_gemini_text(prompt: str, temperature: float, max_output_tokens: int) -> str:
    api_key = _get_api_key()
    if not api_key:
        raise GeminiNotConfiguredError("GEMINI_API_KEY not configured")

    candidate_models = [
        MODEL,
        "gemini-3.8-flash",
        "gemini-2.5-flash",
        "gemini-flash-latest",
        "gemini-1.5-flash",
    ]
    seen = set()
    last_err = None

    for model_name in candidate_models:
        if not model_name or model_name in seen:
            continue
        seen.add(model_name)
        client = None
        try:
```

Code shown in video — timestamp 02:01

```
44    def _call_gemini_text(prompt: str, temperature: float, max_output_tokens: int) -> str:
49        candidate_models = [
50            model,
51            "gemini-3.8-flash",
52            "gemini-2.5-flash",
53            "gemini-flash-latest",
54            "gemini-1.5-flash",
55        ]
56        seen = set()
57        last_err = None
58
59        for model_name in candidate_models:
60            if not model_name or model_name in seen:
61                continue
62            seen.add(model_name)
63            client = None
64            try:
65                client = genai.Client(api_key=api_key)
66                response = client.models.generate_content(
67                    model=model_name,
68                    contents=prompt,
69                    config=types.GenerateContentConfig(
70                        temperature=temperature,
71                        max_output_tokens=max_output_tokens,
72                    ),
73                )
74                text = getattr(response, "text", None)
75                if text and text.strip():
76                    return text.strip()
77            except Exception as exc:
78                last_err = exc
79                logger.warning("Gemini model %s failed: %s", model_name, exc)
```

Code shown in video — timestamp 02:03

```
44    def _call_gemini_text(prompt: str, temperature: float, max_output_tokens: int) -> str:
73                )
74                text = getattr(response, "text", None)
75                if text and text.strip():
76                    return text.strip()
77            except Exception as exc:
78                last_err = exc
79                logger.warning("Gemini model %s failed: %s", model_name, exc)
80                continue
81            finally:
82                if client is not None:
83                    try:
84                        if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
85                            client._api_client._httpx_client.close()
86                    except Exception:
87                        pass
88
89        raise last_err or RuntimeError("Gemini content generation failed.")
90
91
92    def _call_gemini_json(prompt: str, max_output_tokens: int) -> str:
93        api_key = _get_api_key()
94        if not api_key:
95            raise GeminiNotConfiguredError("GEMINI_API_KEY not configured")
96
97        candidate_models = [
98            MODEL,
99            "gemini-3.8-flash",
100           "gemini-2.5-flash",
101           "gemini-flash-latest",
102           "gemini-1.5-flash",
103       ]
```

Code shown in video — timestamp 02:04

```
91
92  def _call_gemini_json(prompt: str, max_output_tokens: int) -> str:
93      api_key = _get_api_key()
94      if not api_key:
95          raise GeminiNotConfiguredError("GEMINI_API_KEY not configured")
96
97      candidate_models = [
98          MODEL,
99          "gemini-3.8-flash",
100         "gemini-2.5-flash",
101         "gemini-flash-latest",
102         "gemini-1.5-flash",
103     ]
104     seen = set()
105     last_err = None
106
107     for model_name in candidate_models:
108         if not model_name or model_name in seen:
109             continue
110         seen.add(model_name)
111         client = None
112         try:
113             client = genai.Client(api_key=api_key)
114             response = client.models.generate_content(
115                 model=model_name,
116                 contents=prompt,
117                 config=types.GenerateContentConfig(
118                     temperature=0.3,
119                     max_output_tokens=max_output_tokens,
120                     response_mime_type="application/json",
121                 ),
```

Code shown in video — timestamp 02:06

```python
def _call_gemini_json(prompt: str, max_output_tokens: int) -> str:
            client = genai.Client(api_key=api_key)
            response = client.models.generate_content(
                model=model_name,
                contents=prompt,
                config=types.GenerateContentConfig(
                    temperature=0.3,
                    max_output_tokens=max_output_tokens,
                    response_mime_type="application/json",
                ),
            )
            text = getattr(response, "text", None)
            if text and text.strip():
                return text.strip()
        except Exception as exc:
            last_err = exc
            logger.warning("Gemini JSON model %s failed: %s", model_name, exc)
            continue
        finally:
            if client is not None:
                try:
                    if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
                        client._api_client._httpx_client.close()
                except Exception:
                    pass

    raise last_err or RuntimeError("Gemini JSON generation failed.")


def generate_text(prompt: str, *, temperature: float = 0.4, max_output_tokens: int = 1200) -> str:
    # 1. Attempt Gemini if API key is present
```

Code shown in video — timestamp 02:07

```
 92    def _call_gemini_json(prompt: str, max_output_tokens: int) -> str:
131                 if client is not None:
132                     try:
133                         if hasattr(client, "_api_client") and hasattr(client._api_client, "_httpx_client"):
134                             client._api_client._httpx_client.close()
135                     except Exception:
136                         pass
137
138         raise last_err or RuntimeError("Gemini JSON generation failed.")
139
140
141     def generate_text(prompt: str, *, temperature: float = 0.4, max_output_tokens: int = 1200) -> str:
142         # 1. Attempt Gemini if API key is present
143         if _get_api_key():
144             try:
145                 return _call_gemini_text(prompt, temperature, max_output_tokens)
146             except Exception as exc:
147                 logger.info("Falling back to local educational generator: %s", exc)
148
149         # 2. Intelligent local educational fallback
150         prompt_lower = prompt.lower()
151         if "student question:" in prompt_lower:
152             match = re.search(r"student question:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
153             question = match.group(1).strip() if match else prompt
154             return fallback_qa(question)
155
156         if "topic:" in prompt_lower and ("educational tutor" in prompt_lower or "explain the topic" in prompt_l
157             match = re.search(r"topic:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
158             topic = match.group(1).strip() if match else prompt
159             return fallback_explain(topic)
160
161         if "passage:" in prompt_lower and "summarize" in prompt_lower:
```

Code shown in video — timestamp 02:09

```
141    def generate_text(prompt: str, *, temperature: float = 0.4, max_output_tokens: int = 1200) -> str:
147                logger.info("Falling back to local educational generator: %s", exc)
148
149        # 2. Intelligent local educational fallback
150        prompt_lower = prompt.lower()
151        if "student question:" in prompt_lower:
152            match = re.search(r"student question:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
153            question = match.group(1).strip() if match else prompt
154            return fallback_qa(question)
155
156        if "topic:" in prompt_lower and ("educational tutor" in prompt_lower or "explain the topic" in prompt_l
157            match = re.search(r"topic:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
158            topic = match.group(1).strip() if match else prompt
159            return fallback_explain(topic)
160
161        if "passage:" in prompt_lower and "summarize" in prompt_lower:
162            match = re.search(r"passage:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
163            passage = match.group(1).strip() if match else prompt
164            return fallback_summary(passage)
165
166        if "curriculum planner" in prompt_lower or "learning path" in prompt_lower:
167            match = re.search(r"topic:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
168            topic = match.group(1).strip() if match else prompt
169            return fallback_learning_path(topic)
170
171        return fallback_qa(prompt)
172
173
174    def generate_json(prompt: str, *, max_output_tokens: int = 1600) -> str:
175        # 1. Attempt Gemini if API key is present
176        if _get_api_key():
177            try:
```

Code shown in video — timestamp 02:10

```
141    def generate_text(prompt: str, *, temperature: float = 0.4, max_output_tokens: int = 1200) -> str:
163            passage = match.group(1).strip() if match else prompt
164            return fallback_summary(passage)
165
166        if "curriculum planner" in prompt_lower or "learning path" in prompt_lower:
167            match = re.search(r"topic:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
168            topic = match.group(1).strip() if match else prompt
169            return fallback_learning_path(topic)
170
171        return fallback_qa(prompt)
172
173
174    def generate_json(prompt: str, *, max_output_tokens: int = 1600) -> str:
175        # 1. Attempt Gemini if API key is present
176        if _get_api_key():
177            try:
178                return _call_gemini_json(prompt, max_output_tokens)
179            except Exception as exc:
180                logger.info("Falling back to local quiz generator: %s", exc)
181
182        # 2. Intelligent quiz fallback
183        match = re.search(r"material:\s*(.+)", prompt, re.IGNORECASE | re.DOTALL)
184        material = match.group(1).strip() if match else prompt
185        return fallback_quiz(material)
186
```

Code shown in video — timestamp 02:12

```python
from gemini_client import generate_text


def get_learning_recommendations(topic: str) -> str:
    topic = topic.strip()
    if not topic:
        raise ValueError("Please enter a learning topic.")

    return generate_text(
        "You are an educational curriculum planner. Create a practical learning path "
        "for the topic below. Organize it from beginner to advanced. Include: "
        "1) stages, 2) suggested time for each stage, 3) what to learn, "
        "4) a small practice task, and 5) useful resource types such as videos, "
        "articles, documentation, or books. Adapt it for a learner who is starting "
        "from the beginning. Do not fabricate specific URLs.\n\n"
        f"Topic: {topic}",
        temperature=0.45,
        max_output_tokens=1500,
    )

```

Code shown in video — timestamp 02:13

```python
def get_learning_recommendations(topic: str) -> str:
    topic = topic.strip()
    if not topic:
        raise ValueError("Please enter a learning topic.")

    return generate_text(
        "You are an educational curriculum planner. Create a practical learning path "
        "for the topic below. Organize it from beginner to advanced. Include: "
        "1) stages, 2) suggested time for each stage, 3) what to learn, "
        "4) a small practice task, and 5) useful resource types such as videos, "
        "articles, documentation, or books. Adapt it for a learner who is starting "
        "from the beginning. Do not fabricate specific URLs.\n\n"
        f"Topic: {topic}",
        temperature=0.45,
        max_output_tokens=1500,
    )

```

Code shown in video — timestamp 02:15

```python

@app.post("/explain")
async def explain(payload: TextRequest):
    try:
        return {"result": explain_concept(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/quiz")
async def quiz(payload: TextRequest):
    try:
        return {"quiz": generate_quiz(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/summarize")
async def summarize(payload: TextRequest):
    try:
        return {"result": summarize_text(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/learn/recommendations")
async def learning_recommendations(payload: TextRequest):
    try:
        return {"result": get_learning_recommendations(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

```

Code shown in video — timestamp 02:16

```python
from pydantic import BaseModel, Field

from explanation_module import explain_concept
from qna import answer_question
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

app = FastAPI(
    title="EduGenie",
    description="Google Gemini powered learning assistant",
    version="1.0.0",
)

from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))


class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=20000)


@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(request=request, name="index.html")


@app.get("/health")
async def health():
```

Code shown in video — timestamp 02:18

```python
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel, Field

from explanation_module import explain_concept
from qna import answer_question
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

app = FastAPI(
    title="EduGenie",
    description="Google Gemini powered learning assistant",
    version="1.0.0",
)

from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))


class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=20000)


@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(request=request, name="index.html")
```

Code shown in video — timestamp 02:19

```
)

from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))


class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=20000)


@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(request=request, name="index.html")


@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie"}


@app.post("/qa")
async def qa(payload: TextRequest):
    try:
        return {"result": answer_question(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/explain")
```

Code shown in video — timestamp 02:21

```python
@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie"}


@app.post("/qa")
async def qa(payload: TextRequest):
    try:
        return {"result": answer_question(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/explain")
async def explain(payload: TextRequest):
    try:
        return {"result": explain_concept(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/quiz")
async def quiz(payload: TextRequest):
    try:
        return {"quiz": generate_quiz(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

Code shown in video — timestamp 02:22

```python

@app.post("/explain")
async def explain(payload: TextRequest):
    try:
        return {"result": explain_concept(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/quiz")
async def quiz(payload: TextRequest):
    try:
        return {"quiz": generate_quiz(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/summarize")
async def summarize(payload: TextRequest):
    try:
        return {"result": summarize_text(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/learn/recommendations")
async def learning_recommendations(payload: TextRequest):
    try:
        return {"result": get_learning_recommendations(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

Code shown in video — timestamp 02:24

```python
async def quiz(payload: TextRequest):
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/summarize")
async def summarize(payload: TextRequest):
    try:
        return {"result": summarize_text(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))


@app.post("/learn/recommendations")
async def learning_recommendations(payload: TextRequest):
    try:
        return {"result": get_learning_recommendations(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

```

Code shown in video — timestamp 02:25

```
57  async def quiz(payload: TextRequest):
60      except Exception as exc:
61          raise HTTPException(status_code=500, detail=str(exc))
62
63
64  @app.post("/summarize")
65  async def summarize(payload: TextRequest):
66      try:
67          return {"result": summarize_text(payload.text)}
68      except Exception as exc:
69          raise HTTPException(status_code=500, detail=str(exc))
70
71
72  @app.post("/learn/recommendations")
73  async def learning_recommendations(payload: TextRequest):
74      try:
75          return {"result": get_learning_recommendations(payload.text)}
76      except Exception as exc:
77          raise HTTPException(status_code=500, detail=str(exc))
78
```

Code shown in video — timestamp 02:27

```
57  async def quiz(payload: TextRequest):
60      except Exception as exc:
61          raise HTTPException(status_code=500, detail=str(exc))
62
63
64  @app.post("/summarize")
65  async def summarize(payload: TextRequest):
66      try:
67          return {"result": summarize_text(payload.text)}
68      except Exception as exc:
69          raise HTTPException(status_code=500, detail=str(exc))
70
71
72  @app.post("/learn/recommendations")
73  async def learning_recommendations(payload: TextRequest):
74      try:
75          return {"result": get_learning_recommendations(payload.text)}
76      except Exception as exc:
77          raise HTTPException(status_code=500, detail=str(exc))
78
```

Code shown in video — timestamp 02:28

```python
def generate_quiz(passage: str) -> List[Dict[str, Any]]:
    passage = passage.strip()
    if not passage:
        raise ValueError("Please provide a topic or passage for the quiz.")

    prompt = f"""
Create exactly 3 educational multiple-choice questions from the material below.

Return ONLY valid JSON with this shape:
{{
  "questions": [
    {{
      "question": "string",
      "options": ["A", "B", "C", "D"],
      "answer": "one exact option string"
    }}
  ]
}}

Rules:
- Exactly 3 questions.
- Exactly 4 options per question.
- One correct answer per question.
- Distractors should be plausible.
- Questions must be answerable from the supplied material.
- Keep language suitable for students.

Material:
{passage}
"""
    raw = generate_json(prompt)
    try:
```

Code shown in video — timestamp 02:30

```python
import json
from typing import List, Dict, Any

from gemini_client import generate_json


def _validate_quiz(data: Any) -> List[Dict[str, Any]]:
    if isinstance(data, dict) and "questions" in data:
        data = data["questions"]

    if not isinstance(data, list) or len(data) != 3:
        raise ValueError("Quiz response must contain exactly 3 questions.")

    clean = []
    for index, item in enumerate(data, start=1):
        if not isinstance(item, dict):
            raise ValueError(f"Question {index} is not an object.")

        question = str(item.get("question", "")).strip()
        options = item.get("options")
        answer = str(item.get("answer", "")).strip()

        if not question or not isinstance(options, list) or len(options) != 4 or not answer:
            raise ValueError(f"Question {index} has an invalid structure.")

        options = [str(option).strip() for option in options]
        if answer not in options:
            raise ValueError(f"Question {index} answer must match one option.")

        clean.append(
            {
                "question": question,
```

Code shown in video — timestamp 02:31

```python
def _validate_quiz(data: Any) -> List[Dict[str, Any]]:
    for index, item in enumerate(data, start=1):
        if not isinstance(item, dict):
            raise ValueError(f"Question {index} is not an object.")

        question = str(item.get("question", "")).strip()
        options = item.get("options")
        answer = str(item.get("answer", "")).strip()

        if not question or not isinstance(options, list) or len(options) != 4 or not answer:
            raise ValueError(f"Question {index} has an invalid structure.")

        options = [str(option).strip() for option in options]
        if answer not in options:
            raise ValueError(f"Question {index} answer must match one option.")

        clean.append(
            {
                "question": question,
                "options": options,
                "answer": answer,
            }
        )
    return clean


def generate_quiz(passage: str) -> List[Dict[str, Any]]:
    passage = passage.strip()
    if not passage:
        raise ValueError("Please provide a topic or passage for the quiz.")

    prompt = f"""
```

Code shown in video — timestamp 02:33

```
def _validate_quiz(data: Any) -> List[Dict[str, Any]]:
            raise ValueError(f"Question {index} answer must match one option.")

        clean.append(
            {
                "question": question,
                "options": options,
                "answer": answer,
            }
        )
    return clean


def generate_quiz(passage: str) -> List[Dict[str, Any]]:
    passage = passage.strip()
    if not passage:
        raise ValueError("Please provide a topic or passage for the quiz.")

    prompt = f"""
Create exactly 3 educational multiple-choice questions from the material below.

Return ONLY valid JSON with this shape:
{{
  "questions": [
    {{
      "question": "string",
      "options": ["A", "B", "C", "D"],
      "answer": "one exact option string"
    }}
  ]
}}
```

Code shown in video — timestamp 02:34

```
40  def generate_quiz(passage: str) -> List[Dict[str, Any]]:
45      prompt = f"""
61  - Exactly 4 options per question.
62  - One correct answer per question.
63  - Distractors should be plausible.
64  - Questions must be answerable from the supplied material.
65  - Keep language suitable for students.
66
67  Material:
68  {passage}
69  """
70      raw = generate_json(prompt)
71      try:
72          parsed = json.loads(raw)
73      except json.JSONDecodeError as exc:
74          raise ValueError("Gemini returned invalid quiz JSON.") from exc
75
76      return _validate_quiz(parsed)
77
```

Code shown in video — timestamp 02:36



Code shown in video — timestamp 02:37

```python
from gemini_client import generate_text


def answer_question(question: str) -> str:
    question = question.strip()
    if not question:
        raise ValueError("Please enter a question.")

    return generate_text(
        "You are EduGenie, an academic question-answering assistant. "
        "Answer the student's question clearly and concisely. "
        "If the question is ambiguous, state the assumption you made. "
        "For calculations, show the key steps. "
        "Do not invent citations or claim certainty when the answer is uncertain.\n\n"
        f"Student question: {question}",
        temperature=0.25,
        max_output_tokens=1000,
    )

```

Code shown in video — timestamp 02:39

```
# EduGenie – Google Gemini Powered Learning Assistant
## Troubleshooting
### Port already in use
Run:

```bash
uvicorn main:app --reload --port 8001
```

Then open `http://127.0.0.1:8001`.

Want API documentation?

FastAPI automatically provides:

```text
http://127.0.0.1:8000/docs
```

and:

```text
http://127.0.0.1:8000/redoc
```
```

Code shown in video — timestamp 02:40

```
# EduGenie – Google Gemini Powered Learning Assistant
## 4. Start the application

The project documentation specifies the same Uvicorn-style local launch and port.

## 5. Test the API

Health:

```bash
curl http://127.0.0.1:8000/health
```

Question answering:

```bash
curl -X POST http://127.0.0.1:8000/qa ^
  -H "Content-Type: application/json" ^
  -d "{\"text\":\"What is photosynthesis?\"}"
```

On macOS/Linux, use `\` instead of `^` for line continuation.

Quiz:

```bash
curl -X POST http://127.0.0.1:8000/quiz \
  -H "Content-Type: application/json" \
  -d '{"text":"The Earth revolves around the Sun. The Moon revolves around Earth."}'
```

Optional local explanation model
```

Code shown in video — timestamp 02:42

```
# EduGenie – Google Gemini Powered Learning Assistant
## Included features
- Educational summarization: `POST /summarize`
- Beginner-to-advanced learning path: `POST /learn/recommendations`
- Responsive single-page web UI
- Health endpoint: `GET /health`

## Project structure

```text
EduGenie/
├── main.py
├── gemini_client.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── requirements-local.txt
├── .env.example
├── .gitignore
├── README.md
├── templates/
│   └── index.html
└── static/
    └── style.css
```

1. Create a virtual environment

Windows
```

Code shown in video — timestamp 02:43

```
# EduGenie – Google Gemini Powered Learning Assistant

EduGenie is a FastAPI + HTML/CSS educational assistant based on the supplied project documentation.

## Included features

- Question answering: `POST /qa`
- Concept explanation: `POST /explain`
- Three-question MCQ quiz generation: `POST /quiz`
- Educational summarization: `POST /summarize`
- Beginner-to-advanced learning path: `POST /learn/recommendations`
- Responsive single-page web UI
- Health endpoint: `GET /health`

## Project structure

```text
EduGenie/
├── main.py
├── gemini_client.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── requirements-local.txt
├── .env.example
├── .gitignore
├── README.md
├── templates/
│   └── index.html
```
```

Code shown in video — timestamp 02:45

```
# EduGenie – Google Gemini Powered Learning Assistant
## Included features
- Beginner-to-advanced learning path: `POST /learn/recommendations`
- Responsive single-page web UI
- Health endpoint: `GET /health`

## Project structure

```text
EduGenie/
├── main.py
├── gemini_client.py
├── explanation_module.py
├── qna.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
├── requirements-local.txt
├── .env.example
├── .gitignore
├── README.md
├── templates/
│   └── index.html
└── static/
    └── style.css
```

1. Create a virtual environment

Windows
```

Code shown in video — timestamp 02:46

```
# EduGenie – Google Gemini Powered Learning Assistant

## 1. Create a virtual environment

### Windows

```powershell
python -m venv .venv
.venv\Scripts\activate
```

macOS / Linux

```bash
python3 -m venv .venv
source .venv/bin/activate
```

2. Install dependencies

```bash
python -m pip install --upgrade pip
pip install -r requirements.txt
```

3. Configure Gemini

Copy `.env.example` to `.env`:

```text
GEMINI_API_KEY=your_real_key
GEMINI_MODEL=gemini-3.8-flash
```
```

Code shown in video — timestamp 02:48

```
# EduGenie – Google Gemini Powered Learning Assistant
## 2. Install dependencies

python -m pip install --upgrade pip
pip install -r requirements.txt
```

3. Configure Gemini

Copy `.env.example` to `.env`:

```text
GEMINI_API_KEY=your_real_key
GEMINI_MODEL=gemini-3.8-flash
EXPLANATION_PROVIDER=gemini
```

The app reads the key from the environment. Never commit `.env` to Git.

4. Start the application

```bash
uvicorn main:app --reload
```

Open:

```text
http://127.0.0.1:8000
```

The project documentation specifies the same Uvicorn-style local launch and port.

Code shown in video — timestamp 02:49

```
# EduGenie – Google Gemini Powered Learning Assistant
## 4. Start the application

```text
http://127.0.0.1:8000
```

The project documentation specifies the same Uvicorn-style local launch and port.

5. Test the API

Health:

```bash
curl http://127.0.0.1:8000/health
```

Question answering:

```bash
curl -X POST http://127.0.0.1:8000/qa ^
  -H "Content-Type: application/json" ^
  -d "{\"text\":\"What is photosynthesis?\"}"
```

On macOS/Linux, use `\` instead of `^` for line continuation.

Quiz:

```bash
curl -X POST http://127.0.0.1:8000/quiz \
  -H "Content-Type: application/json" \
```
```

Code shown in video — timestamp 02:51

```
# EduGenie – Google Gemini Powered Learning Assistant
## 5. Test the API

Quiz:

```bash
curl -X POST http://127.0.0.1:8000/quiz \
  -H "Content-Type: application/json" \
  -d '{"text":"The Earth revolves around the Sun. The Moon revolves around Earth."}'
```

Optional local explanation model

The source document describes LaMini-Flan-T5-783M for concept explanation. This implementation keeps that path available without forcing every installation to download a large ML stack.

Install:

```bash
pip install -r requirements-local.txt
```

Then set:

```text
EXPLANATION_PROVIDER=local
LOCAL_EXPLANATION_MODEL=MBZUAI/LaMini-Flan-T5-783M
```

The first local-model run may download model files and can require substantial disk/RAM.

Troubleshooting
```

Code shown in video — timestamp 02:52

```
# EduGenie – Google Gemini Powered Learning Assistant

## Troubleshooting

### `GEMINI_API_KEY is not configured`

Make sure `.env` exists in the project root and contains a real key. Restart Uvicorn after changing it.

### Gemini model access error

Change `GEMINI_MODEL` to a model available to your API project. The app does not hard-code the model
inside the feature modules.

### Port already in use

Run:

```bash
uvicorn main:app --reload --port 8001
```

Then open `http://127.0.0.1:8001`.

Want API documentation?

FastAPI automatically provides:

```text
http://127.0.0.1:8000/docs
```

and:
```

Code shown in video — timestamp 02:54

```
# EduGenie – Google Gemini Powered Learning Assistant
## Troubleshooting
### Want API documentation?

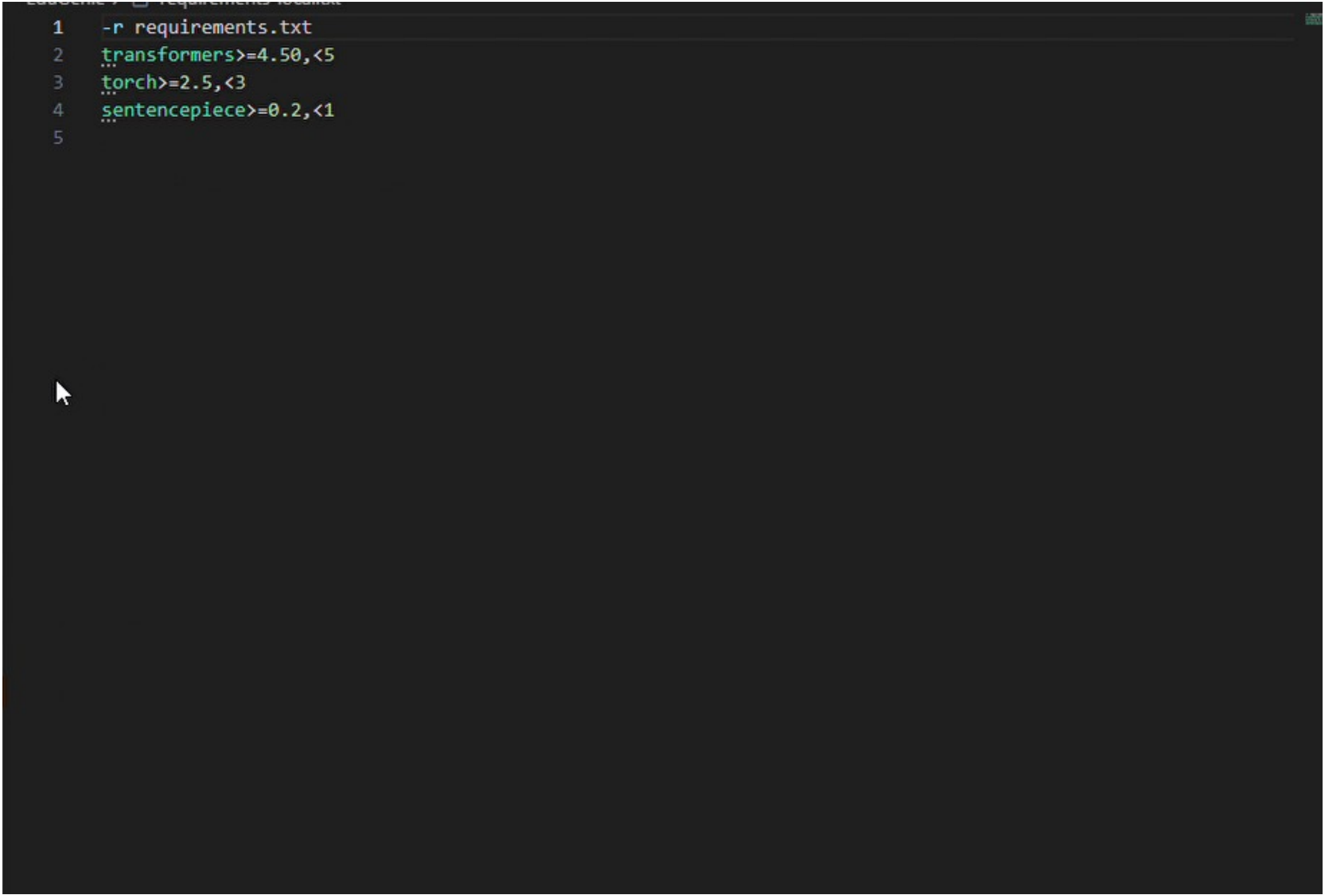
FastAPI automatically provides:

```text
http://127.0.0.1:8000/docs
```

and:

```text
http://127.0.0.1:8000/redoc
```
```

Code shown in video — timestamp 02:55



Code shown in video — timestamp 02:57

```
-r requirements.txt
transformers>=4.50,<5
torch>=2.5,<3
sentencepiece>=0.2,<1

```

Code shown in video — timestamp 02:58

```
pydantic>=2.10,<3
python-dotenv>=1.0,<2
google-genai>=1.40,<2
```

Code shown in video — timestamp 03:00

```
fastapi>=0.115,<1
uvicorn[standard]>=0.34,<1
jinja2>=3.1,<4
pydantic>=2.10,<3
python-dotenv>=1.0,<2
google-genai>=1.40,<2

```

Code shown in video — timestamp 03:01



Code shown in video — timestamp 03:03

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )
```

Code shown in video — timestamp 03:04

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )

```

Code shown in video — timestamp 03:06

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )

```

Code shown in video — timestamp 03:07

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )

```

Code shown in video — timestamp 03:09

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )

```

Code shown in video — timestamp 03:10

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )

```

Code shown in video — timestamp 03:12

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )

```

Code shown in video — timestamp 03:13

```python
from gemini_client import generate_text


def summarize_text(text: str) -> str:
    text = text.strip()
    if not text:
        raise ValueError("Please provide text to summarize.")

    return generate_text(
        "Summarize the educational passage below for quick revision. "
        "Keep the important facts, remove repetition, and use clear bullet points "
        "where helpful. Do not introduce information that is not in the passage.\n\n"
        f"Passage:\n{text}",
        temperature=0.25,
        max_output_tokens=1000,
    )
```